

Human-level control through deep reinforcement learning

Volodymyr Mnih^{1*}, Koray Kavukcuoglu^{1*}, David Silver^{1*}, Andrei A. Rusu¹, Joel Veness¹, Marc G. Bellemare¹, Alex Graves¹, Martin Riedmiller¹, Andreas K. Fiedel¹, Georg Ostrovski¹, Stig Petersen¹, Charles Beattie¹, Amir Sadik¹, Ioannis Antonoglou¹, Helen King¹, Dharmashan Kumaran¹, Daan Wierstra¹, Shane Legg¹ & Demis Hassabis¹

The theory of reinforcement learning provides a normative account¹, deeply rooted in psychological² and neuroscientific³ perspectives on animal behaviour, of how agents may optimize their control of an environment. To use reinforcement learning successfully in situations approaching real-world complexity, however, agents are confronted with a difficult task: they must derive efficient representations of the environment from high-dimensional sensory inputs, and use these to generalize past experience to new situations. Remarkably, humans and other animals seem to solve this problem through a harmonious combination of reinforcement learning and hierarchical sensory processing systems^{4,5}, the former evidenced by a wealth of neural data revealing notable parallels between the phasic signals emitted by dopaminergic neurons and temporal difference reinforcement learning algorithms³. While reinforcement learning agents have achieved some successes in a variety of domains^{6–8}, their applicability has previously been limited to domains in which useful features can be handcrafted, or to domains with fully observed, low-dimensional state spaces. Here we use recent advances in training deep neural networks^{9–11} to develop a novel artificial agent, termed a deep Q-network, that can learn successful policies directly from high-dimensional sensory inputs using end-to-end reinforcement learning. We tested this agent on the challenging domain of classic Atari 2600 games¹². We demonstrate that the deep Q-network agent, receiving only the pixels and the game score as inputs, was able to surpass the performance of all previous algorithms and achieve a level comparable to that of a professional human games tester across a set of 49 games, using the same algorithm, network architecture and hyperparameters. This work bridges the divide between high-dimensional sensory inputs and actions, resulting in the first artificial agent that is capable of learning to excel at a diverse array of challenging tasks.

We set out to create a single algorithm that would be able to develop a wide range of competencies on a varied range of challenging tasks—a central goal of general artificial intelligence¹³ that has eluded previous efforts^{8,14,15}. To achieve this, we developed a novel agent, a deep Q-network (DQN), which is able to combine reinforcement learning with a class of artificial neural network¹⁶ known as deep neural networks. Notably, recent advances in deep neural networks^{9–11}, in which several layers of nodes are used to build up progressively more abstract representations of the data, have made it possible for artificial neural networks to learn concepts such as object categories directly from raw sensory data. We use one particularly successful architecture, the deep convolutional network¹⁷, which uses hierarchical layers of tiled convolutional filters to mimic the effects of receptive fields—inspired by Hubel and Wiesel's seminal work on feedforward processing in early visual cortex¹⁸—thereby exploiting the local spatial correlations present in images, and building in robustness to natural transformations such as changes of viewpoint or scale.

We consider tasks in which the agent interacts with an environment through a sequence of observations, actions and rewards. The goal of the

agent is to select actions in a fashion that maximizes cumulative future reward. More formally, we use a deep convolutional neural network to approximate the optimal action-value function

$$Q^*(s, a) = \max_{\pi} \mathbb{E} [r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots | s_t = s, a_t = a, \pi],$$

which is the maximum sum of rewards r_t discounted by γ at each time-step t , achievable by a behaviour policy $\pi = P(a|s)$, after making an observation (s) and taking an action (a) (see Methods)¹⁹.

Reinforcement learning is known to be unstable or even to diverge when a nonlinear function approximator such as a neural network is used to represent the action-value (also known as Q) function²⁰. This instability has several causes: the correlations present in the sequence of observations, the fact that small updates to Q may significantly change the policy and therefore change the data distribution, and the correlations between the action-values (Q) and the target values $r + \gamma \max_{a'} Q(s', a')$. We address these instabilities with a novel variant of Q-learning, which uses two key ideas. First, we used a biologically inspired mechanism termed experience replay^{21–23} that randomizes over the data, thereby removing correlations in the observation sequence and smoothing over changes in the data distribution (see below for details). Second, we used an iterative update that adjusts the action-values (Q) towards target values that are only periodically updated, thereby reducing correlations with the target.

While other stable methods exist for training neural networks in the reinforcement learning setting, such as neural fitted Q-iteration²⁴, these methods involve the repeated training of networks *de novo* on hundreds of iterations. Consequently, these methods, unlike our algorithm, are too inefficient to be used successfully with large neural networks. We parameterize an approximate value function $Q(s, a; \theta_i)$ using the deep convolutional neural network shown in Fig. 1, in which θ_i are the parameters (that is, weights) of the Q-network at iteration i . To perform experience replay we store the agent's experiences $e_t = (s_t, a_t, r_t, s_{t+1})$ at each time-step t in a data set $D_t = \{e_1, \dots, e_t\}$. During learning, we apply Q-learning updates, on samples (or minibatches) of experience $(s, a, r, s') \sim U(D)$, drawn uniformly at random from the pool of stored samples. The Q-learning update at iteration i uses the following loss function:

$$L_i(\theta_i) = \mathbb{E}_{(s, a, r, s') \sim U(D)} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta_i^-) - Q(s, a; \theta_i) \right)^2 \right]$$

in which γ is the discount factor determining the agent's horizon, θ_i are the parameters of the Q-network at iteration i and θ_i^- are the network parameters used to compute the target at iteration i . The target network parameters θ_i^- are only updated with the Q-network parameters (θ_i) every C steps and are held fixed between individual updates (see Methods).

To evaluate our DQN agent, we took advantage of the Atari 2600 platform, which offers a diverse array of tasks ($n = 49$) designed to be

¹Google DeepMind, 5 New Street Square, London EC4A 3TW, UK.

*These authors contributed equally to this work.

通过深度强化学习实现人性化控制

Volodymyr Mnih^{1*}、Koray Kavukcuoglu^{1*}、David Silver^{1*}、Andrei A. Rusu¹、Joel Veness¹、Marc G. Bellemare¹、Alex Graves¹、Martin Riedmiller¹、Andreas K. Fiedland¹、Georg Ostrovski¹、Stig彼得森¹、查尔斯·贝蒂¹、阿米尔·萨迪克¹、伊奥尼斯·安东诺格鲁¹、海伦·金¹、达尚·库马兰¹、达安·维尔斯特拉¹、肖恩·莱格¹和德米斯·哈萨比斯¹

强化学习理论提供了一种规范性解释¹，深深植根于动物行为的心理学²和神经科学³观点，说明智能体如何优化对环境的控制。然而，为了在接近现实世界复杂性的情况下成功使用强化学习，智能体面临着一项艰巨的任务：他们必须从高维感官输入中获得环境的有效表示，并使用它们将过去的经验推广到新的情况。值得注意的是，人类和其他动物似乎通过强化学习和分层感觉处理系统的和谐结合来解决这个问题^{4,5}，大量的神经数据证明了前者，揭示了多巴胺能神经元发出的相位信号与时间差强化学习算法之间的显著相似之处³。虽然强化学习代理在多个领域取得了一些成功⁶⁻⁸，但它们的适用性此前仅限于可以手工制作有用特征的领域，或具有完全可观察的低维状态空间的领域。在这里，我们利用训练深度神经网络⁹⁻¹¹的最新进展来开发一种新型人工智能体，称为深度Q网络，它可以使用端到端强化学习直接从高维感官输入中学习成功的策略。我们在经典Atari 2600游戏的挑战性领域测试了该代理¹²。我们证明，仅接收像素和游戏得分作为输入的深度Q网络代理能够超越之前所有算法的性能，并使用相同的算法、网络架构和超参数，在49个游戏中达到与专业人类游戏测试员相当的水平。这项工作弥合了高维感官输入和动作之间的鸿沟，从而产生了第一个能够学习并擅长完成各种具有挑战性的任务的人工智能体。

我们着手创建一种单一算法，能够在各种具有挑战性的任务上培养广泛的能力——这是通用人工智能¹³的中心目标，而之前的努力却未能实现^{8,14,15}。为了实现这一目标，我们开发了一种新型代理，即深度Q网络(DQN)，它能够将强化学习与一类称为深度神经网络的人工神经网络¹⁶相结合。值得注意的是，深度神经网络⁹⁻¹¹的最新进展（其中使用多层节点来构建逐渐更抽象的数据表示）使得人工神经网络可以直接从原始感官数据中学习诸如对象类别之类的概念。我们使用一种特别成功的架构，即深度卷积网络¹⁷，它使用平铺卷积滤波器的分层层来模拟感受野的效果（受到Hubel和Wiesel在早期视觉皮层前馈处理方面的开创性工作的启发），从而利用图像中存在的局部空间相关性，并增强对自然变换（例如视点或尺度变化）的鲁棒性。

我们考虑智能体通过一系列观察、行动和奖励与环境交互的任务。的目标

代理是以最大化累积未来奖励的方式选择行动。更正式地说，我们使用深度卷积神经网络来近似最佳动作值函数

$$Q^*(s, a) = \max_{\pi} \mathbb{E}[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots | s_t = s, a_t = a, \pi],$$

这是在进行观察(s)并采取行动(a)后，通过行为策略 $p(a|s)$ 可实现的每个时间步t处由折扣的奖励 r_t 的最大总和（参见方法）¹⁹。

众所周知，当使用神经网络等非线性函数逼近器来表示动作值（也称为Q）函数²⁰时，强化学习会不稳定甚至发散。这种不稳定性有几个原因：观察序列中存在的相关性，Q的小更新可能会显著改变策略，从而改变数据分布，以及动作值(Q)和目标值 $\max_{a'} Q(s, a')$ 之间的相关性。我们通过Q学习的一种新颖变体来解决这些不稳定性，它使用了两个关键思想。首先，我们使用了一种受生物学启发的机制，称为经验重播²¹⁻²³，该机制对数据进行随机化，从而消除观察序列中的相关性并平滑数据分布的变化（详细信息请参见下文）。其次，我们使用迭代更新将动作值(Q)调整为仅定期更新的目标值，从而减少与目标的相关性。

虽然存在其他稳定的方法用于在强化学习设置中训练神经网络，例如神经拟合Q迭代²⁴，但这些方法涉及对网络进行数百次迭代的从头重复训练。因此，与我们的算法不同，这些方法效率太低，无法成功用于大型神经网络。我们使用图1所示的深度卷积神经网络参数化近似值函数 $Q(s, a; h_i)$ ，其中 h_i 是Q网络在迭代i时的参数（即权重）。为了执行经验重放，我们将代理在每个时间步t的经验 $e_t \in \{s_t, a_t, r_t, s_{t+1}\}$ 存储在数据集 $D_t \in \{e_1, \dots, e_t\}$ 中。在学习过程中，我们对从存储样本池中均匀随机抽取的经验样本（或小批量） (s, a, r, s') 、U(D)应用Q学习更新。迭代i处的Q学习更新使用以下损失函数：

$$L_i(\theta_i) = \mathbb{E}_{(s, a, r, s') \sim U(D)} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta_i^-) - Q(s, a; \theta_i) \right)^2 \right]$$

其中 γ 是确定代理视野的折扣因子， h_i 是迭代i时Q网络的参数， h_i^- 是用于计算迭代i时目标的网络参数。目标网络参数 h_i^- 仅在每C个步骤中使用Q网络参数(h_i)进行更新，并且在各个更新之间保持固定（参见方法）。

为了评估我们的DQN代理，我们利用了Atari 2600平台，该平台提供了多种任务(n=49)，旨在

¹Google DeepMind, 5 New Street Square, London EC4A 3TW, UK.

*These authors contributed equally to this work.

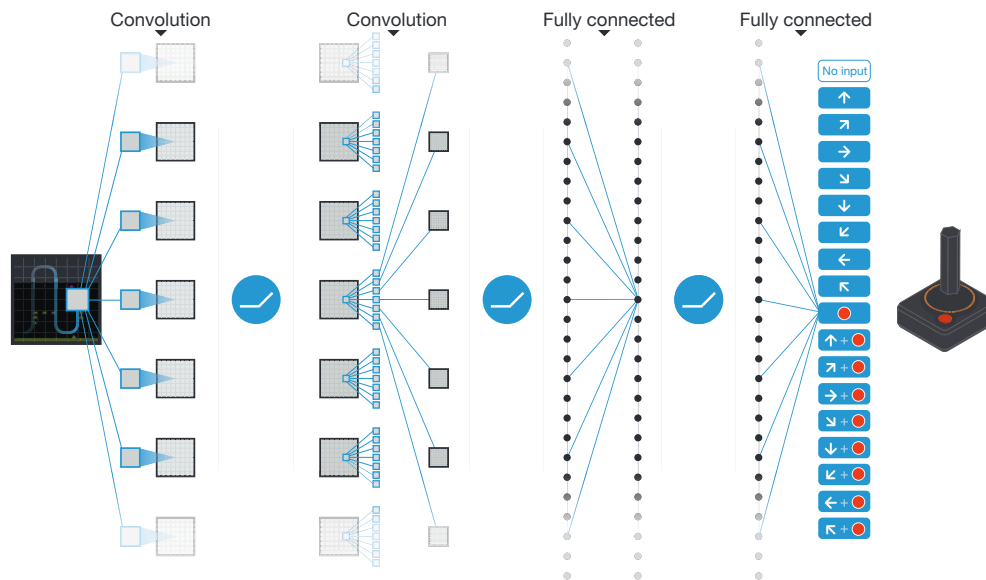


Figure 1 | Schematic illustration of the convolutional neural network. The details of the architecture are explained in the Methods. The input to the neural network consists of an $84 \times 84 \times 4$ image produced by the preprocessing map ϕ , followed by three convolutional layers (note: snaking blue line

symbolizes sliding of each filter across input image) and two fully connected layers with a single output for each valid action. Each hidden layer is followed by a rectifier nonlinearity (that is, $\max(0, x)$).

difficult and engaging for human players. We used the same network architecture, hyperparameter values (see Extended Data Table 1) and learning procedure throughout—taking high-dimensional data (210×160 colour video at 60 Hz) as input—to demonstrate that our approach robustly learns successful policies over a variety of games based solely on sensory inputs with only very minimal prior knowledge (that is, merely the input data were visual images, and the number of actions available in each game, but not their correspondences; see Methods). Notably, our method was able to train large neural networks using a reinforcement learning signal and stochastic gradient descent in a stable manner—illustrated by the temporal evolution of two indices of learning (the agent's average score-per-episode and average predicted Q-values; see Fig. 2 and Supplementary Discussion for details).

We compared DQN with the best performing methods from the reinforcement learning literature on the 49 games where results were available^{12,15}. In addition to the learned agents, we also report scores for a professional human games tester playing under controlled conditions and a policy that selects actions uniformly at random (Extended Data Table 2 and Fig. 3, denoted by 100% (human) and 0% (random) on y axis; see Methods). Our DQN method outperforms the best existing reinforcement learning methods on 43 of the games without incorporating any of the additional prior knowledge about Atari 2600 games used by other approaches (for example, refs 12, 15). Furthermore, our DQN agent performed at a level that was comparable to that of a professional human games tester across the set of 49 games, achieving more than 75% of the human score on more than half of the games (29 games;

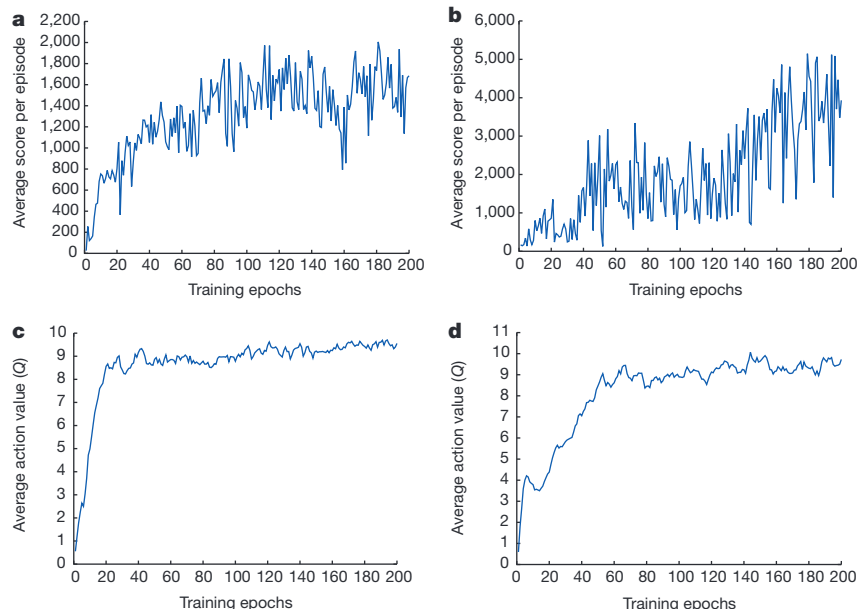


Figure 2 | Training curves tracking the agent's average score and average predicted action-value. **a**, Each point is the average score achieved per episode after the agent is run with ϵ -greedy policy ($\epsilon = 0.05$) for 520 k frames on Space Invaders. **b**, Average score achieved per episode for Seaquest. **c**, Average predicted action-value on a held-out set of states on Space Invaders. Each point

on the curve is the average of the action-value Q computed over the held-out set of states. Note that Q-values are scaled due to clipping of rewards (see Methods). **d**, Average predicted action-value on Seaquest. See Supplementary Discussion for details.

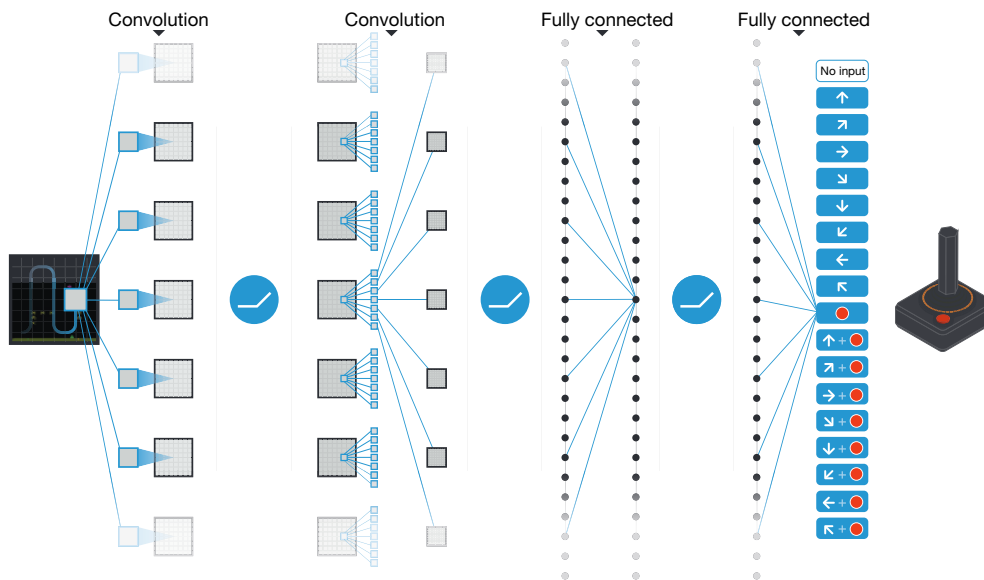


图1|卷积神经网络示意图。方法中解释了架构的细节。神经网络的输入由预处理图 w 生成的 $84 \times 84 \times 3 \times 4$ 图像组成，后面是三个卷积层（注：蜿蜒的蓝线

象征着每个过滤器在输入图像上的滑动）和两个完全连接的层，每个有效动作都有一个输出。每个隐藏层后面都有一个非线性整流器（即 $\max(0, x)$ ）。

对于人类玩家来说既困难又有吸引力。我们使用相同的网络架构、超参数值（参见扩展数据表 1）和学习过程，以高维数据（60 Hz 的 210×160 彩色视频）作为输入，以证明我们的方法仅基于感官输入，仅使用非常少的先验知识（即，仅输入数据是视觉图像和每个游戏中可用的动作数量，而不是它们的对应关系；请参见方法），在各种游戏中稳健地学习成功的策略。值得注意的是，我们的方法能够以稳定的方式使用强化学习信号和随机梯度下降来训练大型神经网络——这可以通过两个学习指数的时间演化来说明（智能体每集的平均得分和平均预测 Q 值；详细信息请参见图 2 和补充讨论）。

我们将 DQN 与强化学习文献中的最佳性能方法进行了比较，涉及 49 款已获得结果的游戏^{12,15}。除了学习到的智能体之外，我们还报告了专业人类游戏测试员在受控条件下玩游戏的分数以及随机均匀选择动作的策略（扩展数据表 2 和图 3，在 y 轴上用 100%（人类）和 0%（随机）表示；参见方法）。我们的 DQN 方法在 43 款游戏中的表现优于现有的最佳强化学习方法，而无需结合其他方法使用的有关 Atari 2600 游戏的任何额外先验知识（例如，参考文献 12、15）。此外，我们的 DQN 代理在 49 款游戏中的表现水平与专业人类游戏测试员的水平相当，在超过一半的游戏中（29 款游戏；29 款游戏；

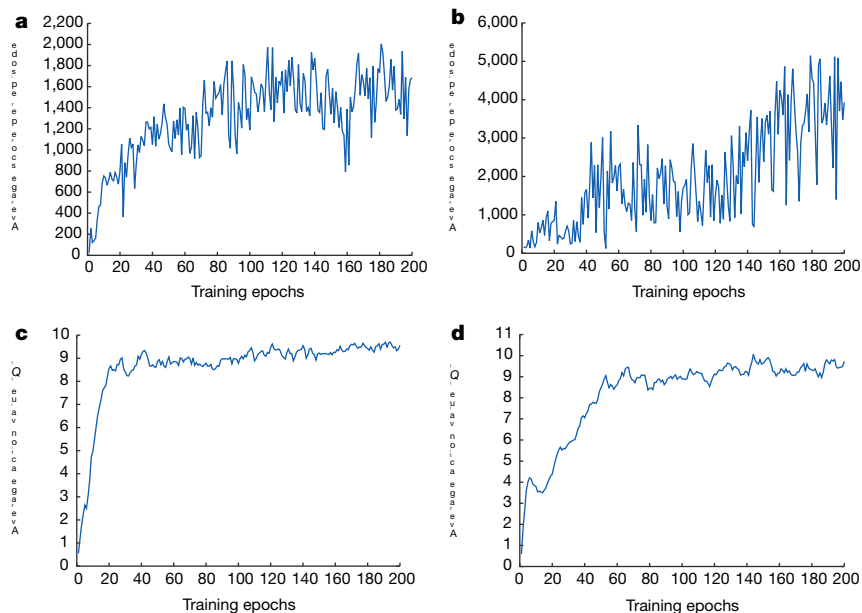


图 2 |跟踪智能体平均得分和平均预测动作值的训练曲线。a，每个点是代理在 Space Invaders 上使用 e-greedy 策略 ($\epsilon = 0.05$) 运行 520 k 帧后每集获得的平均分数。b，Seaquest 每集获得的平均分数。c，《太空入侵者》中一组保留状态的平均预测行动值。每个点

曲线上是保留的状态集上计算出的动作值 Q 的平均值。请注意，由于奖励的削减，Q 值会缩放（请参阅方法）。d，Seaquest 的平均预测行动值。详细信息请参阅补充讨论。

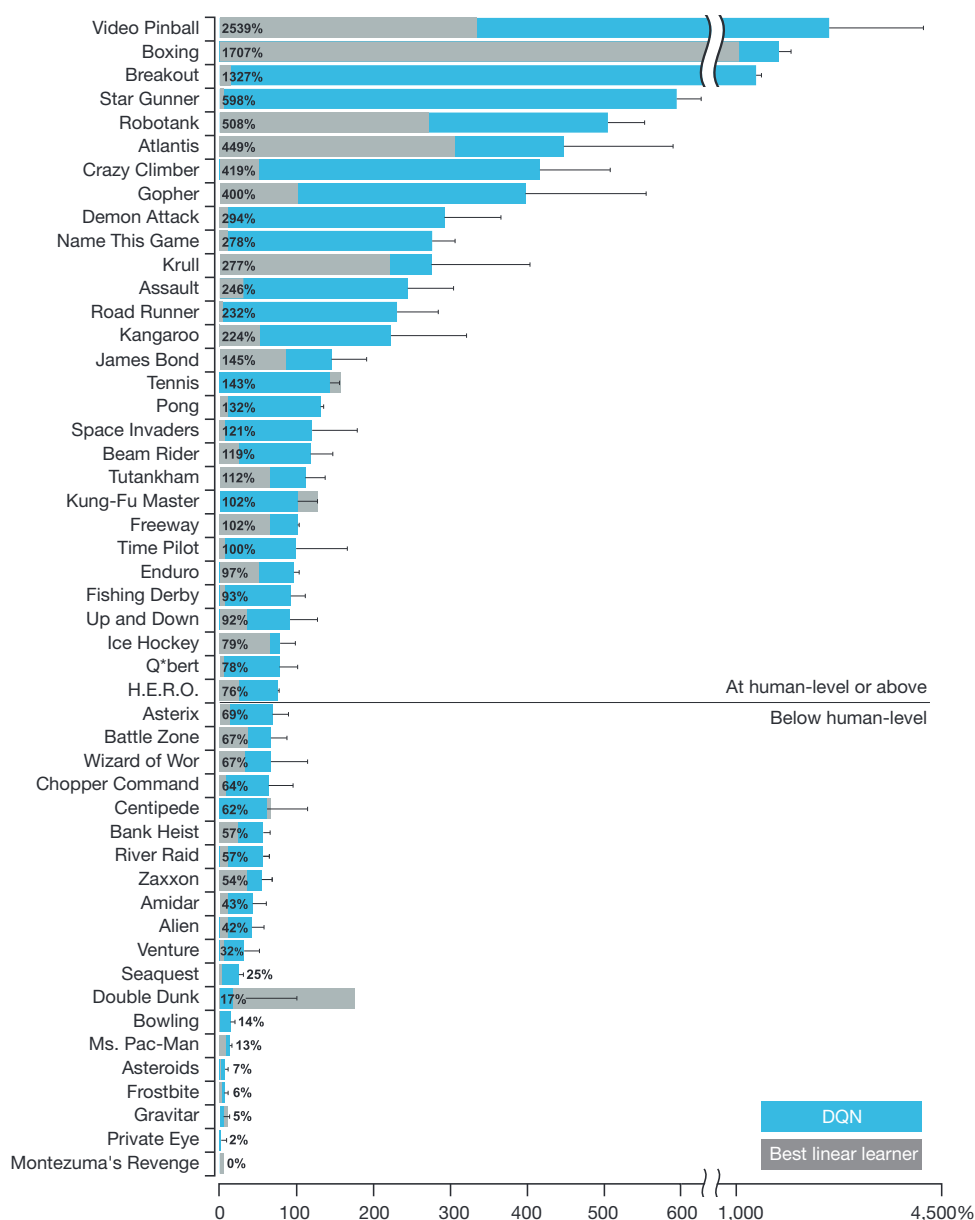


Figure 3 | Comparison of the DQN agent with the best reinforcement learning methods¹⁵ in the literature. The performance of DQN is normalized with respect to a professional human games tester (that is, 100% level) and random play (that is, 0% level). Note that the normalized performance of DQN, expressed as a percentage, is calculated as: $100 \times (\text{DQN score} - \text{random play score}) / (\text{human score} - \text{random play score})$. It can be seen that DQN

outperforms competing methods (also see Extended Data Table 2) in almost all the games, and performs at a level that is broadly comparable with or superior to a professional human games tester (that is, operationalized as a level of 75% or above) in the majority of games. Audio output was disabled for both human players and agents. Error bars indicate s.d. across the 30 evaluation episodes, starting with different initial conditions.

see Fig. 3, Supplementary Discussion and Extended Data Table 2). In additional simulations (see Supplementary Discussion and Extended Data Tables 3 and 4), we demonstrate the importance of the individual core components of the DQN agent—the replay memory, separate target Q-network and deep convolutional network architecture—by disabling them and demonstrating the detrimental effects on performance.

We next examined the representations learned by DQN that underpinned the successful performance of the agent in the context of the game Space Invaders (see Supplementary Video 1 for a demonstration of the performance of DQN), by using a technique developed for the visualization of high-dimensional data called ‘t-SNE’²⁵ (Fig. 4). As expected, the t-SNE algorithm tends to map the DQN representation of perceptually similar states to nearby points. Interestingly, we also found instances in which the t-SNE algorithm generated similar embeddings for DQN representations of states that are close in terms of expected reward but

perceptually dissimilar (Fig. 4, bottom right, top left and middle), consistent with the notion that the network is able to learn representations that support adaptive behaviour from high-dimensional sensory inputs. Furthermore, we also show that the representations learned by DQN are able to generalize to data generated from policies other than its own—in simulations where we presented as input to the network game states experienced during human and agent play, recorded the representations of the last hidden layer, and visualized the embeddings generated by the t-SNE algorithm (Extended Data Fig. 1 and Supplementary Discussion). Extended Data Fig. 2 provides an additional illustration of how the representations learned by DQN allow it to accurately predict state and action values.

It is worth noting that the games in which DQN excels are extremely varied in their nature, from side-scrolling shooters (River Raid) to boxing games (Boxing) and three-dimensional car-racing games (Enduro).

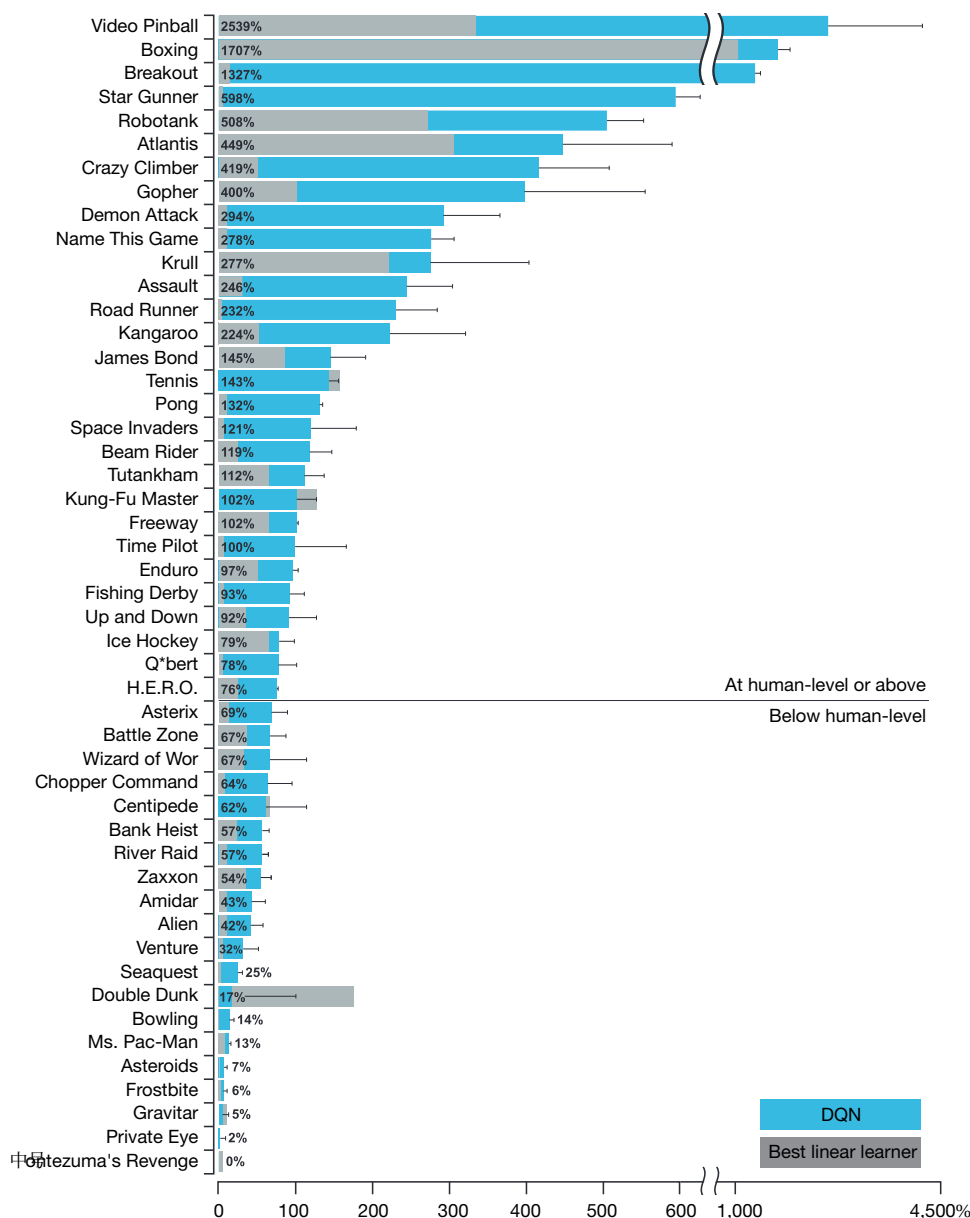


图 3 | DQN 代理与文献中最佳强化学习方法¹⁵ 的比较。DQN 的性能相对于专业人类游戏测试员（即 100% 水平）和随机游戏（即 0% 水平）进行标准化。请注意，DQN 的归一化性能以百分比表示，计算公式为： $100 \times (DQN \text{ 得分} - \text{随机游戏得分}) / (\text{人类得分} - \text{随机游戏得分})$ 。可以看出，DQN 在几乎所有游戏中都优于竞争方法（另见扩展数据表 2），并且在大多数游戏中的表现水平与专业人类游戏测试人员大致相当或优于专业人类游戏测试人员（即，可操作性为 75% 或以上的水平）。人类玩家和代理的音频输出均被禁用。误差线表示 s.d. 跨越 30 个评估阶段，从不同的初始条件开始。

参见图 3，补充讨论和扩展数据表 2）。在其他模拟中（参见补充讨论和扩展数据表 3 和 4），我们通过禁用 DQN 代理的各个核心组件（重播内存、单独的目标 Q 网络和深度卷积网络架构）并展示对性能的不利影响，证明了它们的重要性。

接下来，我们通过使用一种为高维数据可视化开发的技术，称为“t-SNE”²⁵（图 1，检查了 DQN 学习到的表示，这些表示支持代理在游戏《太空入侵者》中的成功表现（请参阅补充视频 1，了解 DQN 的性能演示）。4）。正如预期的那样，t-SNE 算法倾向于将感知相似状态的 DQN 表示映射到附近的点。有趣的是，我们还发现了 t-SNE 算法为状态的 DQN 表示生成类似嵌入的实例，这些状态在预期奖励方面接近，但

感知上不同（图 4，右下，左上和中间），与网络能够从高维感官输入中学习支持自适应行为的表示的概念一致。此外，我们还表明，DQN 学习到的表示能够泛化到从策略生成的数据，而不是它自己的数据 - 在模拟中，我们将其作为人类和代理玩游戏期间经历的网络游戏状态的输入，记录最后一个隐藏层的表示，并可视化 t-SNE 算法生成的嵌入（扩展数据图 1 和补充讨论）。扩展数据图 2 进一步说明了 DQN 学习的表示如何使其能够准确预测状态和动作值。

值得注意的是，DQN 擅长的游戏性质极其多样，从横向卷轴射击游戏（River Raid）到拳击游戏（Boxing）和三维赛车游戏（Enduro）。

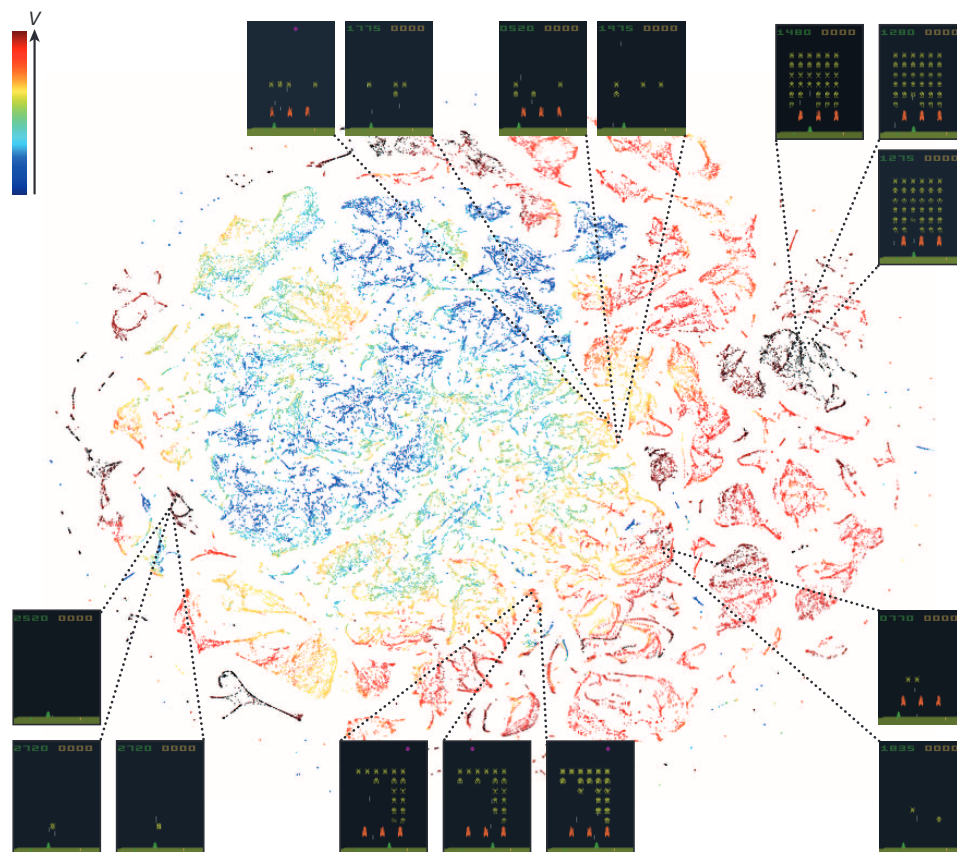


Figure 4 | Two-dimensional t-SNE embedding of the representations in the last hidden layer assigned by DQN to game states experienced while playing Space Invaders. The plot was generated by letting the DQN agent play for 2 h of real game time and running the t-SNE algorithm²⁵ on the last hidden layer representations assigned by DQN to each experienced game state. The points are coloured according to the state values (V , maximum expected reward of a state) predicted by DQN for the corresponding game states (ranging from dark red (highest V) to dark blue (lowest V)). The screenshots corresponding to a selected number of points are shown. The DQN agent

predicts high state values for both full (top right screenshots) and nearly complete screens (bottom left screenshots) because it has learned that completing a screen leads to a new screen full of enemy ships. Partially completed screens (bottom screenshots) are assigned lower state values because less immediate reward is available. The screens shown on the bottom right and top left and middle are less perceptually similar than the other examples but are still mapped to nearby representations and similar values because the orange bunkers do not carry great significance near the end of a level. With permission from Square Enix Limited.

Indeed, in certain games DQN is able to discover a relatively long-term strategy (for example, Breakout: the agent learns the optimal strategy, which is to first dig a tunnel around the side of the wall allowing the ball to be sent around the back to destroy a large number of blocks; see Supplementary Video 2 for illustration of development of DQN's performance over the course of training). Nevertheless, games demanding more temporally extended planning strategies still constitute a major challenge for all existing agents including DQN (for example, Montezuma's Revenge).

In this work, we demonstrate that a single architecture can successfully learn control policies in a range of different environments with only very minimal prior knowledge, receiving only the pixels and the game score as inputs, and using the same algorithm, network architecture and hyperparameters on each game, privy only to the inputs a human player would have. In contrast to previous work^{24,26}, our approach incorporates 'end-to-end' reinforcement learning that uses reward to continuously shape representations within the convolutional network towards salient features of the environment that facilitate value estimation. This principle draws on neurobiological evidence that reward signals during perceptual learning may influence the characteristics of representations within primate visual cortex^{27,28}. Notably, the successful integration of reinforcement learning with deep network architectures was critically dependent on our incorporation of a replay algorithm^{21–23} involving the storage and representation of recently experienced transitions. Convergent evidence suggests that the hippocampus may support the physical

realization of such a process in the mammalian brain, with the time-compressed reactivation of recently experienced trajectories during offline periods^{21,22} (for example, waking rest) providing a putative mechanism by which value functions may be efficiently updated through interactions with the basal ganglia²². In the future, it will be important to explore the potential use of biasing the content of experience replay towards salient events, a phenomenon that characterizes empirically observed hippocampal replay²⁹, and relates to the notion of 'prioritized sweeping'³⁰ in reinforcement learning. Taken together, our work illustrates the power of harnessing state-of-the-art machine learning techniques with biologically inspired mechanisms to create agents that are capable of learning to master a diverse array of challenging tasks.

Online Content Methods, along with any additional Extended Data display items and Source Data, are available in the online version of the paper; references unique to these sections appear only in the online paper.

Received 10 July 2014; accepted 16 January 2015.

1. Sutton, R. & Barto, A. *Reinforcement Learning: An Introduction* (MIT Press, 1998).
2. Thorndike, E. L. *Animal Intelligence: Experimental studies* (Macmillan, 1911).
3. Schultz, W., Dayan, P. & Montague, P. R. A neural substrate of prediction and reward. *Science* **275**, 1593–1599 (1997).
4. Serre, T., Wolf, L. & Poggio, T. Object recognition with features inspired by visual cortex. *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern. Recognit.* 994–1000 (2005).
5. Fukushima, K. Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biol. Cybern.* **36**, 193–202 (1980).

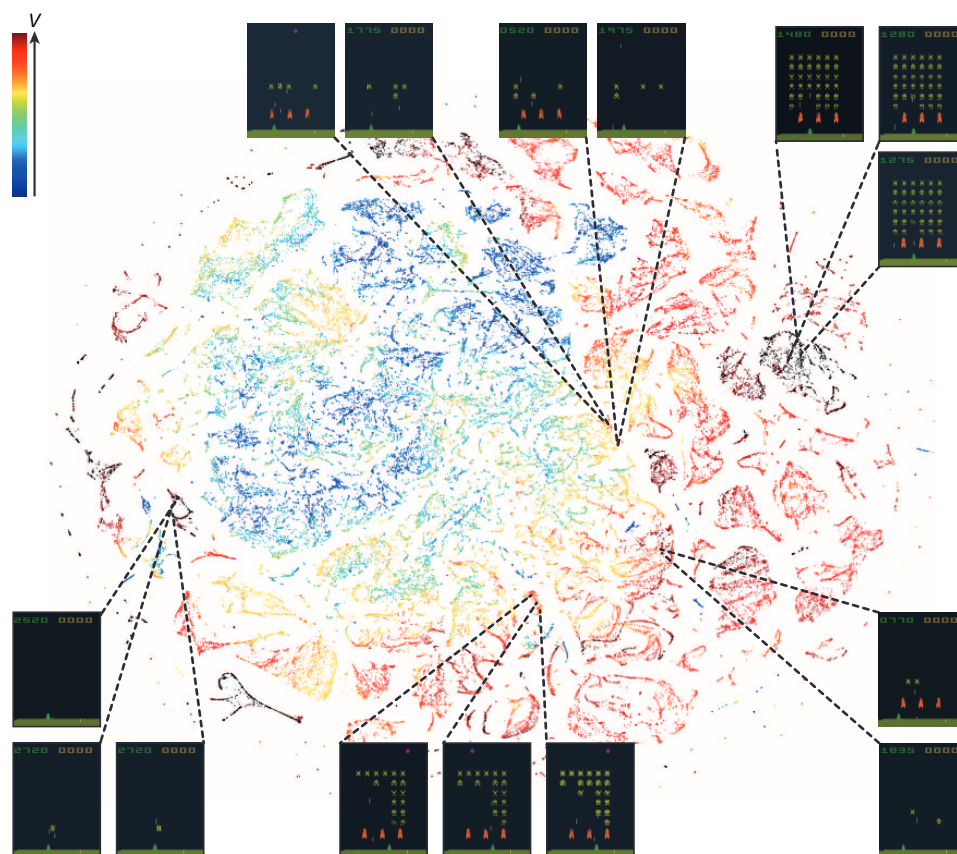


图 4 | 将 DQN 分配的最后一个隐藏层中的表示的二维 t-SNE 嵌入到玩《太空入侵者》时经历的游戏状态。该图是通过让 DQN 代理玩 2 小时的游戏时间并在 DQN 分配给每个经历的游戏状态的最后一个隐藏层表示上运行 t-SNE 算法²⁹来生成的。这些点根据 DQN 预测的相应游戏状态的状态值 (V, 状态的最大预期奖励) 进行着色 (范围从深红色 (最高 V) 到深蓝色 (最低 V))。显示与选定数量的点相对应的屏幕截图。DQN 代理预测完整屏幕 (右上屏幕截图) 和接近完整屏幕 (左下屏幕截图) 的高状态值, 因为它了解到完成屏幕会导致充满敌方船只的新屏幕。部分完成的屏幕 (底部屏幕截图) 被分配较低的状态值, 因为可获得的即时奖励较少。右下角、左上角和中间显示的屏幕在感知上与其他示例相比不太相似, 但仍然映射到附近的表示和类似的值, 因为橙色掩体在关卡末尾附近并不具有很大的意义。经史克威尔艾尼克斯有限公司许可。

事实上, 在某些游戏中, DQN 能够发现相对长期的策略 (例如, Breakout: 智能体学习最佳策略, 即首先在墙边挖一条隧道, 让球从后面发送以摧毁大量方块; 有关 DQN 在训练过程中性能发展的说明, 请参阅补充视频²)。尽管如此, 需要更多时间扩展规划策略的游戏仍然对包括 DQN 在内的所有现有智能体构成重大挑战 (例如, 蒙特祖玛的复仇)。

在这项工作中, 我们证明了单一架构可以在一系列不同环境中成功学习控制策略, 只需非常少的先验知识, 仅接收像素和游戏分数作为输入, 并在每个游戏上使用相同的算法、网络架构和超参数, 仅了解人类玩家可能拥有的输入。与之前的工作^{24,26}相比, 我们的方法结合了“端到端”强化学习, 利用奖励不断塑造卷积网络内的表示, 以实现有助于价值估计的环境显着特征。这一原理借鉴了神经生物学证据, 即感知学习期间的奖励信号可能会影响灵长类动物视觉皮层^{27,28}内的表征特征。值得注意的是, 强化学习与深度网络架构的成功集成在很大程度上取决于我们对重播算法²¹⁻²³的结合, 该算法涉及最近经历的转换的存储和表示。一致的证据表明, 海马体可能支持身体功能。

在哺乳动物大脑中实现这一过程, 通过在离线期间对最近经历的轨迹进行时间压缩重新激活^{21,22} (例如, 清醒休息) 提供了一种假定的机制, 通过该机制可以通过与基底神经节的相互作用来有效地更新价值函数²²。未来, 探索将经验重放内容偏向显着事件的潜在用途将很重要, 这种现象是经验观察到的海马重放的特征²⁹, 并且与强化学习中“优先清扫”³⁰的概念相关。总而言之, 我们的工作展示了利用最先进的机器学习技术和受生物学启发的机制来创建能够学习掌握各种具有挑战性的任务的智能体的力量。

在线内容方法以及任何附加的扩展数据显示项和源数据可在本文的在线版本中找到; 这些部分特有的参考文献仅出现在在线论文中。

2014 年 7 月 10 日收稿; 2015 年 1 月 16 日接受。

1. Sutton, R. 和 Barto, A. 强化学习: 简介 (麻省理工学院出版社, 1998 年)。2. Thorndike, E. L. 《动物智力: 实验研究》(Macmillan, 1911)。3. Schultz, W., Dayan, P. 和 Montague, P. R. 预测和奖励的神经基础。科学 275, 1593–1599 (1997)。4. Serre, T., Wolf, L. 和 Poggio, T. 受视觉皮层启发的物体识别功能。过程。IEEE。计算。苏克。会议。计算。维多。图案。认识。994–1000 (2005)。5. Fukushima, K. Neocognitron: 一种自组织神经网络模型, 用于不受位置变化影响的模式识别机制。Biol. Cybern. 36, 193–202 (1980)。

6. Tesauro, G. Temporal difference learning and TD-Gammon. *Commun. ACM* **38**, 58–68 (1995).
7. Riedmiller, M., Gabel, T., Hafner, R. & Lange, S. Reinforcement learning for robot soccer. *Auton. Robots* **27**, 55–73 (2009).
8. Diuk, C., Cohen, A. & Littman, M. L. An object-oriented representation for efficient reinforcement learning. *Proc. Int. Conf. Mach. Learn.* 240–247 (2008).
9. Bengio, Y. Learning deep architectures for AI. *Foundations and Trends in Machine Learning* **2**, 1–127 (2009).
10. Krizhevsky, A., Sutskever, I. & Hinton, G. ImageNet classification with deep convolutional neural networks. *Adv. Neural Inf. Process. Syst.* **25**, 1106–1114 (2012).
11. Hinton, G. E. & Salakhutdinov, R. R. Reducing the dimensionality of data with neural networks. *Science* **313**, 504–507 (2006).
12. Bellemare, M. G., Naddaf, Y., Veness, J. & Bowling, M. The arcade learning environment: An evaluation platform for general agents. *J. Artif. Intell. Res.* **47**, 253–279 (2013).
13. Legg, S. & Hutter, M. Universal Intelligence: a definition of machine intelligence. *Minds Mach.* **17**, 391–444 (2007).
14. Genesereth, M., Love, N. & Pell, B. General game playing: overview of the AAAI competition. *AI Mag.* **26**, 62–72 (2005).
15. Bellemare, M. G., Veness, J. & Bowling, M. Investigating contingency awareness using Atari 2600 games. *Proc. Conf. AAAI. Artif. Intell.* 864–871 (2012).
16. McClelland, J. L., Rumelhart, D. E. & Group, T. P. R. *Parallel Distributed Processing: Explorations in the Microstructure of Cognition* (MIT Press, 1986).
17. LeCun, Y., Bottou, L., Bengio, Y. & Haffner, P. Gradient-based learning applied to document recognition. *Proc. IEEE* **86**, 2278–2324 (1998).
18. Hubel, D. H. & Wiesel, T. N. Shape and arrangement of columns in cat's striate cortex. *J. Physiol.* **165**, 559–568 (1963).
19. Watkins, C. J. & Dayan, P. Q-learning. *Mach. Learn.* **8**, 279–292 (1992).
20. Tsitsiklis, J. & Roy, B. V. An analysis of temporal-difference learning with function approximation. *IEEE Trans. Automat. Contr.* **42**, 674–690 (1997).
21. McClelland, J. L., McNaughton, B. L. & O'Reilly, R. C. Why there are complementary learning systems in the hippocampus and neocortex: insights from the successes and failures of connectionist models of learning and memory. *Psychol. Rev.* **102**, 419–457 (1995).
22. O'Neill, J., Pleydell-Bouverie, B., Dupret, D. & Csicsvari, J. Play it again: reactivation of waking experience and memory. *Trends Neurosci.* **33**, 220–229 (2010).
23. Lin, L.-J. Reinforcement learning for robots using neural networks. Technical Report, DTIC Document (1993).
24. Riedmiller, M. Neural fitted Q iteration - first experiences with a data efficient neural reinforcement learning method. *Mach. Learn.: ECML* **3720**, 317–328 (Springer, 2005).
25. Van der Maaten, L. J. P. & Hinton, G. E. Visualizing high-dimensional data using t-SNE. *J. Mach. Learn. Res.* **9**, 2579–2605 (2008).
26. Lange, S. & Riedmiller, M. Deep auto-encoder neural networks in reinforcement learning. *Proc. Int. Jt. Conf. Neural Netw.* 1–8 (2010).
27. Law, C.-T. & Gold, J. I. Reinforcement learning can account for associative and perceptual learning on a visual decision task. *Nature Neurosci.* **12**, 655 (2009).
28. Sigala, N. & Logothetis, N. K. Visual categorization shapes feature selectivity in the primate temporal cortex. *Nature* **415**, 318–320 (2002).
29. Bendor, D. & Wilson, M. A. Biasing the content of hippocampal replay during sleep. *Nature Neurosci.* **15**, 1439–1444 (2012).
30. Moore, A. & Atkeson, C. Prioritized sweeping: reinforcement learning with less data and less real time. *Mach. Learn.* **13**, 103–130 (1993).

Supplementary Information is available in the online version of the paper.

Acknowledgements We thank G. Hinton, P. Dayan and M. Bowling for discussions, A. Cain and J. Keene for work on the visuals, K. Keller and P. Rogers for help with the visuals, G. Wayne for comments on an earlier version of the manuscript, and the rest of the DeepMind team for their support, ideas and encouragement.

Author Contributions V.M., K.K., D.S., J.V., M.G.B., M.R., A.G., D.W., S.L. and D.H. conceptualized the problem and the technical framework. V.M., K.K., A.A.R. and D.S. developed and tested the algorithms. J.V., S.P., C.B., A.A.R., M.G.B., I.A., A.K.F., G.O. and A.S. created the testing platform. K.K., H.K., S.L. and D.H. managed the project. K.K., D.K., D.H., V.M., D.S., A.G., A.A.R., J.V. and M.G.B. wrote the paper.

Author Information Reprints and permissions information is available at www.nature.com/reprints. The authors declare no competing financial interests. Readers are welcome to comment on the online version of the paper. Correspondence and requests for materials should be addressed to K.K. (korayk@google.com) or D.H. (demishassabis@google.com).

6. Tesauro, G. 时间差分学习和 TD-Gammon. 交流. *ACM* 38, 58–68 (1995).
7. Riedmiller, M., Gabel, T., Hafner, R. 和 Lange, S. 机器人足球的强化学习. *奥顿. 机器人* 27, 55–73 (2009).
8. Diuk, C., Cohen, A. & Littman, M. L. 高效强化学习的面向对象表示. *过程. 国际. 会议. 马赫. 学习.* 240–247 (2008).
9. Bengio, Y. 学习人工智能的深度架构. *机器学习的基础和趋势* 2, 1–127 (2009).
10. Krizhevsky, A., Sutskever, I. 和 Hinton, G. 使用深度卷积神经网络进行 ImageNet 分类. *高级神经信息处理系统* 25, 1106–1114 (2012).
11. Hinton, G. E. 和 Salakhutdinov, R. R. 使用神经网络降低数据维度. *科学* 313, 504–507 (2006).
12. Bellemare, M. G., Naddaf, Y., Veness, J. 和 Bowling, M. 街机学习环境: 一般代理的评估平台. *J. 阿蒂夫. 英特尔. 资源.* 47, 253–279 (2013).
13. Legg, S. & Hutter, M. 通用智能: 机器智能的定义. *头脑马赫.* 17, 391–444 (2007).
14. Genesereth, M., Love, N. 和 Pell, B. 一般游戏玩法: AAAI 竞赛概述. *人工智能杂志.* 26, 62–72 (2005).
15. Bellemare, M. G., Veness, J. 和 Bowling, M. 使用 Atari 2600 游戏调查应急意识. *过程. 会议. A AAI. 阿蒂夫. 英特尔.* 864–871 (2012).
16. McClelland, J. L., Rumelhart, D. E. & Group, T. P. R. 并行分布式处理: 认知微观结构的探索 (麻省理工学院出版社, 1986 年).
17. LeCun, Y., Bottou, L., Bengio, Y. 和 Haffner, P. 基于梯度的学习应用于文档识别. *过程. IEEE* 86, 2278–2324 (1998).
18. Hubel, D. H. & Wiesel, T. N. 猫纹状皮层柱的形状和排列. *J. 生理学.* 165, 559–568 (1963).
19. Watkins, C. J. 和 Dayan, P. Q-learning. *马赫. 学习.* 8, 279–292 (1992).
20. Tsitsiklis, J. & Roy, B. V. 使用函数逼近的时间差异学习分析. *IEEE 传输. 自动机. 控制.* 42, 674–690 (1997).
21. McClelland, J. L., McNaughton, B. L. & O’Reilly, R. C. 为什么海马体和新皮质中存在互补的学习系统: 从学习和记忆的联结主义模型的成功和失败中获得的见解. *心理. 修订版* 102, 419–457 (1995).
22. O’Neill, J., Pleydell-Bouverie, B., Dupret, D. 和 Csicsvari, J. 再玩一遍: 清醒体验和记忆的重新激活. *神经科学趋势.* 33, 220–229 (2010).
23. 林, L.-J. 使用神经网络的机器人强化学习. 技术报告, DTIC 文件 (1993).
24. Riedmiller, M. 神经拟合 Q 迭代 - 数据高效神经强化学习方法的首次体验. *马赫. 学习: ECML*, 3720, 317–328 (Springer, 2005).
25. Van der Maaten, L. J. P. 和 Hinton, G. E. 使用 t-SNE 可视化高维数据. *J. 马赫. 学习. 资源.* 9, 2579–2605 (2008).
26. Lange, S. & Riedmiller, M. 强化学习中的深度自动编码器神经网络. *过程. 国际. Jt. 会议. 神经性的. 网络.* 1–8 (2010).
27. 劳, C.-T. & Gold, J. I. 强化学习可以解释视觉决策任务中的联想和感知学习. *自然神经科学.* 12, 655 (2009).
28. Sigala, N. & Logothetis, N. K. 视觉分类形状具有灵长类颞叶皮层的选择性. *自然* 415, 318–320 (2002).
29. Bendor, D. & Wilson, M. A. 睡眠期间海马重放内容的偏差. *自然神经科学.* 15, 1439–1444 (2012).
30. Moore, A. 和 Atkeson, C. 优先清扫: 使用较少数据和较少实时时间的强化学习. *马赫. 学习.* 13, 103–130 (1993).

补充信息可在本文的在线版本中找到。

致谢 我们感谢 G. Hinton、P. Dayan 和 M. Bowling 的讨论, 感谢 A. Cain 和 J. Keene 在视觉效果方面的工作, 感谢 K. Keller 和 P. Rogers 在视觉效果方面的帮助, G. Wainwright 对早期版本手稿的评论, 以及 DeepMind 团队其他成员的支持、想法和鼓励。

作者贡献 V.M., K.K., D.S., J.V., M.G.B., M.R., A.G., D.W., S.L. D.H. 概念化了问题和技术框架. V.M., K.K., A.A.R. D.S. 开发并测试了算法. J.V., S.P., C. B., A.A.R., M.G.B., I.A., A.K.F., G.O. 和 A.S. 创建了测试平台. K.K., H.K., S. L. D.H. 负责管理该项目. K.K., D.K., D.H., V.M., D.S., A.G., A.A.R., J.V. 和 M.G.B. 论文写道。

作者信息 转载和许可信息可在 www.nature.com/reprints 上获取。作者声明不存在竞争的经济利益。欢迎读者对论文网络版发表评论。信件和材料请求应发送至 K.K. (k.orayk@google.com) 或 D.H. (demishassabis@google.com)。

METHODS

Preprocessing. Working directly with raw Atari 2600 frames, which are 210×160 pixel images with a 128-colour palette, can be demanding in terms of computation and memory requirements. We apply a basic preprocessing step aimed at reducing the input dimensionality and dealing with some artefacts of the Atari 2600 emulator. First, to encode a single frame we take the maximum value for each pixel colour value over the frame being encoded and the previous frame. This was necessary to remove flickering that is present in games where some objects appear only in even frames while other objects appear only in odd frames, an artefact caused by the limited number of sprites Atari 2600 can display at once. Second, we then extract the Y channel, also known as luminance, from the RGB frame and rescale it to 84×84 . The function ϕ from algorithm 1 described below applies this preprocessing to the m most recent frames and stacks them to produce the input to the Q-function, in which $m = 4$, although the algorithm is robust to different values of m (for example, 3 or 5).

Code availability. The source code can be accessed at <https://sites.google.com/a/deepmind.com/dqn> for non-commercial uses only.

Model architecture. There are several possible ways of parameterizing Q using a neural network. Because Q maps history-action pairs to scalar estimates of their Q-value, the history and the action have been used as inputs to the neural network by some previous approaches^{24,26}. The main drawback of this type of architecture is that a separate forward pass is required to compute the Q-value of each action, resulting in a cost that scales linearly with the number of actions. We instead use an architecture in which there is a separate output unit for each possible action, and only the state representation is an input to the neural network. The outputs correspond to the predicted Q-values of the individual actions for the input state. The main advantage of this type of architecture is the ability to compute Q-values for all possible actions in a given state with only a single forward pass through the network.

The exact architecture, shown schematically in Fig. 1, is as follows. The input to the neural network consists of an $84 \times 84 \times 4$ image produced by the preprocessing map ϕ . The first hidden layer convolves 32 filters of 8×8 with stride 4 with the input image and applies a rectifier nonlinearity^{31,32}. The second hidden layer convolves 64 filters of 4×4 with stride 2, again followed by a rectifier nonlinearity. This is followed by a third convolutional layer that convolves 64 filters of 3×3 with stride 1 followed by a rectifier. The final hidden layer is fully-connected and consists of 512 rectifier units. The output layer is a fully-connected linear layer with a single output for each valid action. The number of valid actions varied between 4 and 18 on the games we considered.

Training details. We performed experiments on 49 Atari 2600 games where results were available for all other comparable methods^{12,15}. A different network was trained on each game: the same network architecture, learning algorithm and hyperparameter settings (see Extended Data Table 1) were used across all games, showing that our approach is robust enough to work on a variety of games while incorporating only minimal prior knowledge (see below). While we evaluated our agents on unmodified games, we made one change to the reward structure of the games during training only. As the scale of scores varies greatly from game to game, we clipped all positive rewards at 1 and all negative rewards at -1 , leaving 0 rewards unchanged. Clipping the rewards in this manner limits the scale of the error derivatives and makes it easier to use the same learning rate across multiple games. At the same time, it could affect the performance of our agent since it cannot differentiate between rewards of different magnitude. For games where there is a life counter, the Atari 2600 emulator also sends the number of lives left in the game, which is then used to mark the end of an episode during training.

In these experiments, we used the RMSProp (see http://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf) algorithm with minibatches of size 32. The behaviour policy during training was ϵ -greedy with ϵ annealed linearly from 1.0 to 0.1 over the first million frames, and fixed at 0.1 thereafter. We trained for a total of 50 million frames (that is, around 38 days of game experience in total) and used a replay memory of 1 million most recent frames.

Following previous approaches to playing Atari 2600 games, we also use a simple frame-skipping technique¹⁵. More precisely, the agent sees and selects actions on every k th frame instead of every frame, and its last action is repeated on skipped frames. Because running the emulator forward for one step requires much less computation than having the agent select an action, this technique allows the agent to play roughly k times more games without significantly increasing the runtime. We use $k = 4$ for all games.

The values of all the hyperparameters and optimization parameters were selected by performing an informal search on the games Pong, Breakout, Seaquest, Space Invaders and Beam Rider. We did not perform a systematic grid search owing to the high computational cost. These parameters were then held fixed across all other games. The values and descriptions of all hyperparameters are provided in Extended Data Table 1.

Our experimental setup amounts to using the following minimal prior knowledge: that the input data consisted of visual images (motivating our use of a convolutional deep network), the game-specific score (with no modification), number of actions, although not their correspondences (for example, specification of the up 'button') and the life count.

Evaluation procedure. The trained agents were evaluated by playing each game 30 times for up to 5 min each time with different initial random conditions ('no-op'; see Extended Data Table 1) and an ϵ -greedy policy with $\epsilon = 0.05$. This procedure is adopted to minimize the possibility of overfitting during evaluation. The random agent served as a baseline comparison and chose a random action at 10 Hz which is every sixth frame, repeating its last action on intervening frames. 10 Hz is about the fastest that a human player can select the 'fire' button, and setting the random agent to this frequency avoids spurious baseline scores in a handful of the games. We did also assess the performance of a random agent that selected an action at 60 Hz (that is, every frame). This had a minimal effect: changing the normalized DQN performance by more than 5% in only six games (Boxing, Breakout, Crazy Climber, Demon Attack, Krull and Robotank), and in all these games DQN outperformed the expert human by a considerable margin.

The professional human tester used the same emulator engine as the agents, and played under controlled conditions. The human tester was not allowed to pause, save or reload games. As in the original Atari 2600 environment, the emulator was run at 60 Hz and the audio output was disabled: as such, the sensory input was equated between human player and agents. The human performance is the average reward achieved from around 20 episodes of each game lasting a maximum of 5 min each, following around 2 h of practice playing each game.

Algorithm. We consider tasks in which an agent interacts with an environment, in this case the Atari emulator, in a sequence of actions, observations and rewards. At each time-step the agent selects an action a_t from the set of legal game actions, $\mathcal{A} = \{1, \dots, K\}$. The action is passed to the emulator and modifies its internal state and the game score. In general the environment may be stochastic. The emulator's internal state is not observed by the agent; instead the agent observes an image $x_t \in \mathbb{R}^d$ from the emulator, which is a vector of pixel values representing the current screen. In addition it receives a reward r_t representing the change in game score. Note that in general the game score may depend on the whole previous sequence of actions and observations; feedback about an action may only be received after many thousands of time-steps have elapsed.

Because the agent only observes the current screen, the task is partially observed³³ and many emulator states are perceptually aliased (that is, it is impossible to fully understand the current situation from only the current screen x_t). Therefore, sequences of actions and observations, $s_t = x_1, a_1, x_2, \dots, a_{t-1}, x_t$, are input to the algorithm, which then learns game strategies depending upon these sequences. All sequences in the emulator are assumed to terminate in a finite number of time-steps. This formalism gives rise to a large but finite Markov decision process (MDP) in which each sequence is a distinct state. As a result, we can apply standard reinforcement learning methods for MDPs, simply by using the complete sequence s_t as the state representation at time t .

The goal of the agent is to interact with the emulator by selecting actions in a way that maximizes future rewards. We make the standard assumption that future rewards are discounted by a factor of γ per time-step (γ was set to 0.99 throughout), and

define the future discounted return at time t as $R_t = \sum_{t'=t}^T \gamma^{t'-t} r_{t'}$, in which T is the

time-step at which the game terminates. We define the optimal action-value function $Q^*(s, a)$ as the maximum expected return achievable by following any policy, after seeing some sequence s and then taking some action a , $Q^*(s, a) = \max_{\pi} \mathbb{E}[R_t | s_t = s, a_t = a, \pi]$ in which π is a policy mapping sequences to actions (or distributions over actions).

The optimal action-value function obeys an important identity known as the Bellman equation. This is based on the following intuition: if the optimal value $Q^*(s', a')$ of the sequence s' at the next time-step was known for all possible actions a' , then the optimal strategy is to select the action a' maximizing the expected value of $r + \gamma Q^*(s', a')$:

$$Q^*(s, a) = \mathbb{E}_s \left[r + \gamma \max_{a'} Q^*(s', a') \mathbb{B} a \right]$$

The basic idea behind many reinforcement learning algorithms is to estimate the action-value function by using the Bellman equation as an iterative update, $Q_{i+1}(s, a) = \mathbb{E}_s [r + \gamma \max_{a'} Q_i(s', a') \mathbb{B} a]$. Such value iteration algorithms converge to the optimal action-value function, $Q_i \rightarrow Q^*$ as $i \rightarrow \infty$. In practice, this basic approach is impractical, because the action-value function is estimated separately for each sequence, without any generalization. Instead, it is common to use a function approximator to estimate the action-value function, $Q(s, a; \theta) \approx Q^*(s, a)$. In the reinforcement learning community this is typically a linear function approximator, but

方法

预处理。直接处理原始 Atari 2600 帧（即具有 128 色调色板的 $210 \times 316 \times 0$ 像素图像）对计算和内存的要求可能很高。我们应用了一个基本的预处理步骤，旨在减少输入维度并处理 Atari 2600 模拟器的一些问题。首先，为了对单个帧进行编码，我们采用正在编码的帧和前一帧上每个像素颜色值的最大值。这是消除游戏中出现的闪烁所必需的，游戏中某些对象仅出现在偶数帧中，而其他对象仅出现在奇数帧中，这是由于 Atari 2600 可以同时显示的精灵数量有限而造成的现象。其次，我们从 RGB 帧中提取 Y 通道（也称为亮度），并将其重新调整为 84×84 。下面描述的算法 1 中的函数 w 将此预处理应用于最近的 m 个帧，并将它们堆叠以生成 Q 函数的输入，其中 $m=5$ ，尽管该算法对于不同的 m 值（例如 3 或 5）具有鲁棒性。

代码可用性。源代码可在 <https://sites.google.com/a/deeppmind.com/dqn> 访问，仅供非商业用途。

模型架构。使用神经网络参数化 Q 有多种可能的方法。由于 Q 将历史动作对映射到其 Q 值的标量估计，因此历史和动作已被一些先前的方法用作神经网络的输入^{24,26}。这种类型的架构的主要缺点是需要单独的前向传递来计算每个动作的 Q 值，从而导致成本随动作数量线性扩展。相反，我们使用一种架构，其中每个可能的动作都有一个单独的输出单元，并且只有状态表示是神经网络的输入。输出对应于输入状态的各个动作的预测 Q 值。这种类型的架构的主要优点是能够计算给定状态下所有可能动作的 Q 值，而无需通过网络进行一次前向传递。

确切的架构如图 1 所示，如下所示。神经网络的输入由预处理映射 w 生成的 $84 \times 84 \times 3$ 图像组成。第一个隐藏层将 8×8 个步长为 4 的 32 个滤波器与输入图像进行卷积，并应用整流器非线性^{31,32}。第二个隐藏层将 $64 \times 4 \times 4$ 的滤波器与步幅 2 进行卷积，后面同样是一个非线性整流器。接下来是第三个卷积层，它对 $3 \times 3 \times 3$ 的 64 个滤波器进行卷积，步长为 1，后面是一个整流器。最后的隐藏层是全连接的，由 512 个整流器单元组成。输出层是一个全连接的线性层，每个有效动作都有一个输出。在我们考虑的游戏，有效动作的数量在 4 到 18 之间变化。

培训细节。我们对 49 款 Atari 2600 游戏进行了实验，其中的结果适用于所有其他类似方法^{12,15}。每个游戏都训练了不同的网络：所有游戏都使用相同的网络架构、学习算法和超参数设置（参见扩展数据表 1），这表明我们的方法足够强大，可以在各种游戏上工作，同时只结合最少的先验知识（参见下文）。虽然我们在未修改的游戏上评估我们的代理，但我们仅在训练期间对游戏的奖励结构进行了一项更改。由于不同游戏的分数大小差异很大，我们将所有正奖励削减为 1，将所有负奖励削减为 -21，保留 0 奖励不变。以这种方式削减奖励可以限制误差导数的规模，并且可以更轻松地多个游戏中使用相同的学习率。同时，它可能会影响我们代理的表现，因为它无法区分不同大小的奖励。对于有生命计数器的游戏，Atari 2600 模拟器还会发送游戏中剩余的生命数，然后用于在训练期间标记剧集的结束。

在这些实验中，我们使用了 RMSProp（参见 http://www.cs.toronto.edu/tijmen/csc321/slides/lecture_slides_lec6.pdf）算法，小批量大小为 32。训练期间的行为策略是 e-greedy，e 在前一百万帧中从 1.0 线性退火到 0.1，此后固定为 0.1。我们总共训练了 5000 万帧（即总共约 38 天的游戏体验），并使用了最近 100 万帧的重放内存。

遵循之前玩 Atari 2600 游戏的方法，我们还使用了简单的跳帧技术¹⁵。更准确地说，代理在每个第 k 帧而不是每一帧上查看并选择动作，并且在跳过的帧上重复其最后一个动作。由于向前运行模拟器一步所需的计算量比让代理选择动作要少得多，因此该技术允许代理玩大约 k 倍的游戏，而不会显著增加运行时间。我们对所有游戏都使用 $k=5$ 。

所有超参数和优化参数的值都是通过通过对 Pong、Breakout、Seaquest、Space Invaders 和 Beam Rider 游戏进行非正式搜索来选择的。由于计算成本较高，我们没有进行系统的网格搜索。然后，这些参数在所有其他游戏中保持固定。扩展数据表 1 中提供了所有超参数的值和描述。

我们的实验设置相当于使用以下最少的先验知识：输入数据由视觉图像（激发我们使用卷积深度网络）、特定于游戏的得分（未经修改）、动作数量（尽管不是它们的对应关系）（例如，向上“按钮”的规范）和生命计数器组成。

评估程序。通过使用不同的初始随机条件（“noop”；参见扩展数据表 1）和 $\epsilon=0.05$ 的 e-greedy 策略玩每个游戏 30 次，每次最多 5 分钟，对经过训练的智能体进行评估。采用该程序是为了最大限度地减少评估过程中过度拟合的可能性。随机代理作为基线比较，并选择每六帧一次的 10 Hz 随机动作，在中间帧上重复其最后一个动作。10 Hz 大约是人类玩家选择“开火”按钮的最快速度，将随机代理设置为该频率可以避免少数游戏中出现虚假基线分数。我们还评估了以 60 Hz（即每帧）选择动作的随机代理的性能。这产生的影响很小：仅在六款游戏（《拳击》、《突破》、《疯狂攀登者》、《恶魔攻击》、《克鲁尔》和《机器人坦克》）中，标准化 DQN 性能就改变了 5% 以上，并且在所有这些游戏中，DQN 的表现都远远超过了人类专家。

专业的人体测试人员使用与代理相同的模拟器引擎，并在受控条件下进行游戏。人类测试人员不得暂停、保存或重新加载游戏。与原始 Atari 2600 环境中一样，模拟器以 60 Hz 运行，并且禁用音频输出：因此，人类玩家和代理之间的感官输入是等同的。人类表现是每场游戏大约 20 集、每场游戏持续最多 5 分钟、每场游戏进行大约 2 小时练习后获得的平均奖励。

算法。我们考虑代理与环境（在本例中为 Atari 模拟器）交互的任务，包括一系列动作、观察和奖励。在每个时间步，代理从合法游戏动作集中选择一个动作 a_t ， $A=1, \dots, K$ 。该动作被传递到模拟器并修改其内部状态和游戏分数。一般来说，环境可能是随机的。代理无法观察到模拟器的内部状态；相反，代理从模拟器观察图像 $x_t \in \mathbb{R}^d$ ，它是表示当前屏幕的像素值向量。此外，它还收到代表游戏得分变化的奖励 r_t 。请注意，一般来说，游戏得分可能取决于之前的整个动作和观察序列；有关某个操作的反馈可能只有在经过数千个时间步之后才能收到。

由于代理仅观察当前屏幕，因此任务被部分观察³³，并且许多模拟器状态在感知上存在别名（即仅从当前屏幕 x_t 无法完全了解当前情况）。因此，动作和观察序列 $s_t = x_1, a_1, x_2, \dots, a_{t-1}, x_t$ 被输入到算法中，然后算法根据这些序列学习游戏策略。假设仿真器中的所有序列都在有限数量的时间步长内终止。这种形式主义产生了一个大型但有限的马尔可夫决策过程（MDP），其中每个序列都是一个不同的状态。因此，我们可以将标准的强化学习方法应用于 MDP，只需使用完整的序列 s_t 作为时间 t 的状态表示即可。

代理的目标是通过以最大化未来奖励的方式选择操作来与模拟器进行交互。我们做出标准假设，即未来奖励每个时间步按 c 因子折扣（ c 始终设置为 0.99），并将时间 t 处的未来折扣回报定义为 $R = \sum_{i=0}^{\infty} c^i r_{t+i}$ ，其中 T 是游戏终止的时间步。我们将最优动作价值函数 $Q^*(s, a)$ 定义为在看到一些序列 s 然后采取一些动作 a ， $Q^*(s, a) = \max_{\pi} \mathbb{E}_\pi [R | s, a]$ 后，遵循任何策略可实现的最大预期回报其中 p 是将序列映射到动作（或动作上的分布）的策略。

期望

最佳行动价值函数遵循一个重要的恒等式，称为贝尔曼方程。这是基于以下直觉：如果对于所有可能的动作 a ，序列 s 在下一时间步的最优值 $Q^*(s', a)$ 已知，则最优策略是选择动作 a 最大化 $Q^*(s, a)$ 的期望值：

$$Q^*(s, a) = \mathbb{E}_\pi \left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right]$$

许多强化学习算法背后的基本思想是通过使用贝尔曼方程作为迭代更新来估计动作价值函数， $Q_{t+1}(s, a) = \gamma \mathbb{E}_\pi [r + \max_{a'} Q_t(s', a') \mid s, a]$ 。这种价值迭代算法收敛于最佳动作价值函数 Q^* ？ Q^* 是我吗？？在实践中，这种基本方法是不切实际的，因为动作价值函数是针对每个序列单独估计的，没有任何概括。相反，通常使用函数逼近器来估计动作价值函数 $Q(s, a)$ ； $h \delta p \approx Q^*(s, a)$ 在强化学习社区中，这通常是线性函数逼近器，但是

sometimes a nonlinear function approximator is used instead, such as a neural network. We refer to a neural network function approximator with weights θ as a Q-network. A Q-network can be trained by adjusting the parameters θ_i at iteration i to reduce the mean-squared error in the Bellman equation, where the optimal target values $r + \gamma \max_{a'} Q^*(s', a')$ are substituted with approximate target values $y = r + \gamma \max_{a'} Q(s', a'; \theta_i^-)$, using parameters θ_i^- from some previous iteration. This leads to a sequence of loss functions $L_i(\theta_i)$ that changes at each iteration i ,

$$L_i(\theta_i) = \mathbb{E}_{s,a,r} [(\mathbb{E}_{s'} [y|s,a] - Q(s,a; \theta_i))^2] \\ = \mathbb{E}_{s,a,r,s'} [(y - Q(s,a; \theta_i))^2] + \mathbb{E}_{s,a,r} [V_{s'}[y]] :$$

Note that the targets depend on the network weights; this is in contrast with the targets used for supervised learning, which are fixed before learning begins. At each stage of optimization, we hold the parameters from the previous iteration θ_i^- fixed when optimizing the i th loss function $L_i(\theta_i)$, resulting in a sequence of well-defined optimization problems. The final term is the variance of the targets, which does not depend on the parameters θ_i that we are currently optimizing, and may therefore be ignored. Differentiating the loss function with respect to the weights we arrive at the following gradient:

$$+_{\theta_i} L(\theta_i) = \mathbb{E}_{s,a,r,s'} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta_i^-) - Q(s,a; \theta_i) \right) +_{\theta_i} Q(s,a; \theta_i) \right] :$$

Rather than computing the full expectations in the above gradient, it is often computationally expedient to optimize the loss function by stochastic gradient descent. The familiar Q-learning algorithm¹⁹ can be recovered in this framework by updating the weights after every time step, replacing the expectations using single samples, and setting $\theta_i^- = \theta_{i-1}$.

Note that this algorithm is model-free: it solves the reinforcement learning task directly using samples from the emulator, without explicitly estimating the reward and transition dynamics $P(r, s' | s, a)$. It is also off-policy: it learns about the greedy policy $a = \arg \max_{a'} Q(s, a'; \theta)$, while following a behaviour distribution that ensures adequate exploration of the state space. In practice, the behaviour distribution is often selected by an ε -greedy policy that follows the greedy policy with probability $1 - \varepsilon$ and selects a random action with probability ε .

Training algorithm for deep Q-networks. The full algorithm for training deep Q-networks is presented in Algorithm 1. The agent selects and executes actions according to an ε -greedy policy based on Q . Because using histories of arbitrary length as inputs to a neural network can be difficult, our Q-function instead works on a fixed length representation of histories produced by the function ϕ described above. The algorithm modifies standard online Q-learning in two ways to make it suitable for training large neural networks without diverging.

First, we use a technique known as experience replay²³ in which we store the agent's experiences at each time-step, $e_t = (s_t, a_t, r_t, s_{t+1})$, in a data set $D_t = \{e_1, \dots, e_t\}$, pooled over many episodes (where the end of an episode occurs when a terminal state is reached) into a replay memory. During the inner loop of the algorithm, we apply Q-learning updates, or minibatch updates, to samples of experience, $(s, a, r, s') \sim U(D)$, drawn at random from the pool of stored samples. This approach has several advantages over standard online Q-learning. First, each step of experience is potentially used in many weight updates, which allows for greater data efficiency. Second, learning directly from consecutive samples is inefficient, owing to the strong correlations between the samples; randomizing the samples breaks these correlations and therefore reduces the variance of the updates. Third, when learning on-policy the current parameters determine the next data sample that the parameters are trained on. For example, if the maximizing action is to move left then the training samples will be dominated by samples from the left-hand side; if the maximizing action then switches to the right then the training distribution will also switch. It is easy to see how unwanted feedback loops may arise and the parameters could get stuck in a poor local minimum, or even diverge catastrophically²⁰. By using experience

replay the behaviour distribution is averaged over many of its previous states, smoothing out learning and avoiding oscillations or divergence in the parameters. Note that when learning by experience replay, it is necessary to learn off-policy (because our current parameters are different to those used to generate the sample), which motivates the choice of Q-learning.

In practice, our algorithm only stores the last N experience tuples in the replay memory, and samples uniformly at random from D when performing updates. This approach is in some respects limited because the memory buffer does not differentiate important transitions and always overwrites with recent transitions owing to the finite memory size N . Similarly, the uniform sampling gives equal importance to all transitions in the replay memory. A more sophisticated sampling strategy might emphasize transitions from which we can learn the most, similar to prioritized sweeping³⁰.

The second modification to online Q-learning aimed at further improving the stability of our method with neural networks is to use a separate network for generating the targets y_j in the Q-learning update. More precisely, every C updates we clone the network Q to obtain a target network $\hat{Q}$ and use $\hat{Q}$ for generating the Q-learning targets y_j for the following C updates to Q . This modification makes the algorithm more stable compared to standard online Q-learning, where an update that increases $Q(s_t, a_t)$ often also increases $Q(s_{t+1}, a)$ for all a and hence also increases the target y_t , possibly leading to oscillations or divergence of the policy. Generating the targets using an older set of parameters adds a delay between the time an update to Q is made and the time the update affects the targets y_j , making divergence or oscillations much more unlikely.

We also found it helpful to clip the error term from the update $r + \gamma \max_{a'} Q(s', a'; \theta_i^-) - Q(s, a; \theta_i)$ to be between -1 and 1 . Because the absolute value loss function $|x|$ has a derivative of -1 for all negative values of x and a derivative of 1 for all positive values of x , clipping the squared error to be between -1 and 1 corresponds to using an absolute value loss function for errors outside of the $(-1, 1)$ interval. This form of error clipping further improved the stability of the algorithm.

Algorithm 1: deep Q-learning with experience replay.

Initialize replay memory D to capacity N

Initialize action-value function Q with random weights θ

Initialize target action-value function $\hat{Q}$ with weights $\theta^- = \theta$

For episode = 1, M **do**

Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$

For $t = 1, T$ **do**

With probability ε select a random action a_t

otherwise select $a_t = \arg \max_a Q(\phi(s_t), a; \theta)$

Execute action a_t in emulator and observe reward r_t and image x_{t+1}

Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D

Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D

Set $y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$

Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ with respect to the network parameters θ

Every C steps reset $\hat{Q} = Q$

End For

End For

31. Jarrett, K., Kavukcuoglu, K., Ranzato, M. A. & LeCun, Y. What is the best multi-stage architecture for object recognition? *Proc. IEEE. Int. Conf. Comput. Vis.* 2146–2153 (2009).
32. Nair, V. & Hinton, G. E. Rectified linear units improve restricted Boltzmann machines. *Proc. Int. Conf. Mach. Learn.* 807–814 (2010).
33. Kaelbling, L. P., Littman, M. L. & Cassandra, A. R. Planning and acting in partially observable stochastic domains. *Artificial Intelligence* **101**, 99–134 (1994).

有时会使用非线性函数逼近器，例如神经网络。我们将权重为 h 的神经网络函数逼近器称为 Q 网络。可以通过在迭代 i 处调整参数 h_i 来训练 Q 网络，以减少贝尔曼方程中的均方误差，其中最优化目标值 $\text{rzc max}_{a'} Q(s, a'; h_i^-)$ 替换为近似目标值 $y = \text{rzc max}_{a'} Q(s, a'; h_i^-)$ ，使用之前迭代中的参数 h_i^- 。这导致一系列损失函数 $L_i(h_i)$ 在每次迭代 i 时发生变化，

$$L_i(\theta_i) = \mathbb{E}_{s,a,r} [(E_{s'}[y|D,a] - Q(s,a; \theta_i))^2] \\ = \mathbb{E}_{s,a,r,s'} [(y - Q(s,a; \theta_i))^2] + E_{s,a,r} [V_{s'}[y]] :$$

请注意，目标取决于网络权重；这与监督学习使用的目标形成鲜明对比，后者在学习开始之前就已确定。在优化的每个阶段，我们在优化第 i 个损失函数 $L_i(h_i)$ 时保持前一次迭代 h_i^- 的参数固定，从而产生一系列明确定义的优化问题。最后一项是目标的方差，它不依赖于我们当前正在优化的参数 h_i ，因此可以被忽略。将损失函数与权重微分，我们得到以下梯度：

$$+\theta_i L(\theta_i) = \mathbb{E}_{s,a,r,s'} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta_i^-) - Q(s,a; \theta_i) \right) + \theta_i Q(s,a; \theta_i) \right] :$$

通过随机梯度下降来优化损失函数通常在计算上是方便的，而不是计算上述梯度中的全部期望。通过在每个时间步后更新权重、使用单个样本替换期望以及设置 $h_i^- = h_{i-1}$ ，可以在此框架中恢复熟悉的 Q 学习算法¹⁹。

请注意，该算法是无模型的：它直接使用模拟器中的样本来解决强化学习任务，而无需明确估计奖励和转换动态 $P(r, s'|D, s, a)$ 。它也是离策略的：它学习贪婪策略 $a = \arg\max_{a'} Q(s, a'; h_i^-)$ ，同时遵循确保对状态空间进行充分探索的行为分布。在实践中，行为分布通常是通过 ϵ 贪婪策略来选择的，该策略遵循概率为 $1 - \epsilon$ 的贪婪策略并选择概率为 ϵ 的随机动作。

深度 Q 网络的训练算法。算法 1 中介绍了训练深度 Q 网络的完整算法。代理根据基于 Q 的 ϵ -greedy 策略选择和执行操作。因为使用任意长度的历史作为神经网络的输入可能很困难，所以我们的 Q 函数改为使用上述函数 w 生成的历史的固定长度表示。该算法以两种方式修改了标准在线 Q 学习，使其适合训练大型神经网络而不发散。

首先，我们使用一种称为经验重播²³的技术，其中我们将代理在每个时间步的经验 $e_t \in (s_t, a_t, r_t, s_{t+1})$ 存储在数据集 $D_t \in \{e_1, \dots, e_t\}$ 中，汇集在许多情节中（其中情节的结束发生在达到终端状态）进入重播存储器。在算法的内循环期间，我们将 Q 学习更新或小批量更新应用于从存储样本池中随机抽取的经验样本 (s, a, r, s') ， $U(D)$ 。与标准在线 Q 学习相比，这种方法有几个优点。首先，每一步的经验都可能用于许多权重更新，从而提高数据效率。其次，由于样本之间的相关性很强，直接从连续样本中学习效率很低；随机化样本会破坏这些相关性，从而减少更新的方差。第三，当按策略学习时，当前参数决定参数训练的下一个数据样本。例如，如果最大化动作是向左移动，那么训练样本将以左侧的样本为主；如果最大化动作随后切换到右侧，那么训练分布也会切换。很容易看出可能会出现不需要的反馈循环，并且参数可能会陷入不良的局部最小值，甚至灾难性地发散²⁰。通过使用经验

重放行为分布是对其之前的许多状态进行平均，从而平滑学习并避免参数的振荡或发散。请注意，当通过经验回放进行学习时，有必要学习离策略（因为我们当前的参数与用于生成样本的参数不同），这激发了 Q -learning 的选择。

实际上，我们的算法仅将最后 N 个经验元组存储在重播内存中，并在执行更新时从 D 中均匀随机采样。这种方法在某些方面受到限制，因为内存缓冲区不区分重要的转换，并且由于内存大小 N 有限，总是用最近的转换覆盖。类似地，均匀采样对重放内存中的所有转换给予同等的重要性。更复杂的采样策略可能会强调我们可以从中学到最多的转换，类似于优先扫描²⁰。

对在线 Q 学习的第二个修改旨在进一步提高神经网络方法的稳定性，即使用单独的网络在 Q 学习更新中生成目标 y_j 。更准确地说，每次 C 更新时，我们都会克隆网络 Q 以获得目标网络 $\hat{Q}$ ，并使用 $\hat{Q}$ 生成 Q 学习目标 y_j ，以便随后对 Q 进行 C 更新。与标准在线 Q 学习相比，这种修改使算法更加稳定，其中增加 $Q(s_t, a_t)$ 的更新通常也会增加所有 a 的 $Q(s_{t+1}, a)$ ，因此也增加了目标 y_j ，可能导致政策的振荡或发散。使用较旧的参数集生成目标会在 Q 更新时间与更新影响目标 y_j 之间增加延迟，从而更不可能出现发散或振荡。

我们还发现从更新 $\text{rzc max}_{a'} Q(s, a'; h_i^-)$ 中剪辑误差项很有帮助。 $h_i^- \bar{y} \{Q(s, a; h_i^-) - \bar{y}\}$ 在 21 和 1 之间。因为绝对值损失函数 $|x|$ 对于 x 的所有负值都有 21 的导数，对于 x 的所有正值都有 1 的导数，因此将平方误差限制在 21 和 1 之间相当于对 $(21, 1)$ 区间之外的误差使用绝对值损失函数。这种形式的误差裁剪进一步提高了算法的稳定性。

算法 1: 带有经验回放的深度 Q 学习。

将重放存储器 D 初始化为容量 N

使用随机权重 h 初始化动作价值函数 Q 使用权重 h^- 初始化目标动作价值函数 $\hat{Q}$

佛 r episode = 1, M do

Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$

For $t = 1, T$ do

With probability ϵ select a random action a_t

otherwise select $a_t = \arg\max_a Q(\phi(s_t), a; \theta)$

Execute action a_t in emulator and observe reward r_t and image x_{t+1}

Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D

Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D

Set $y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$

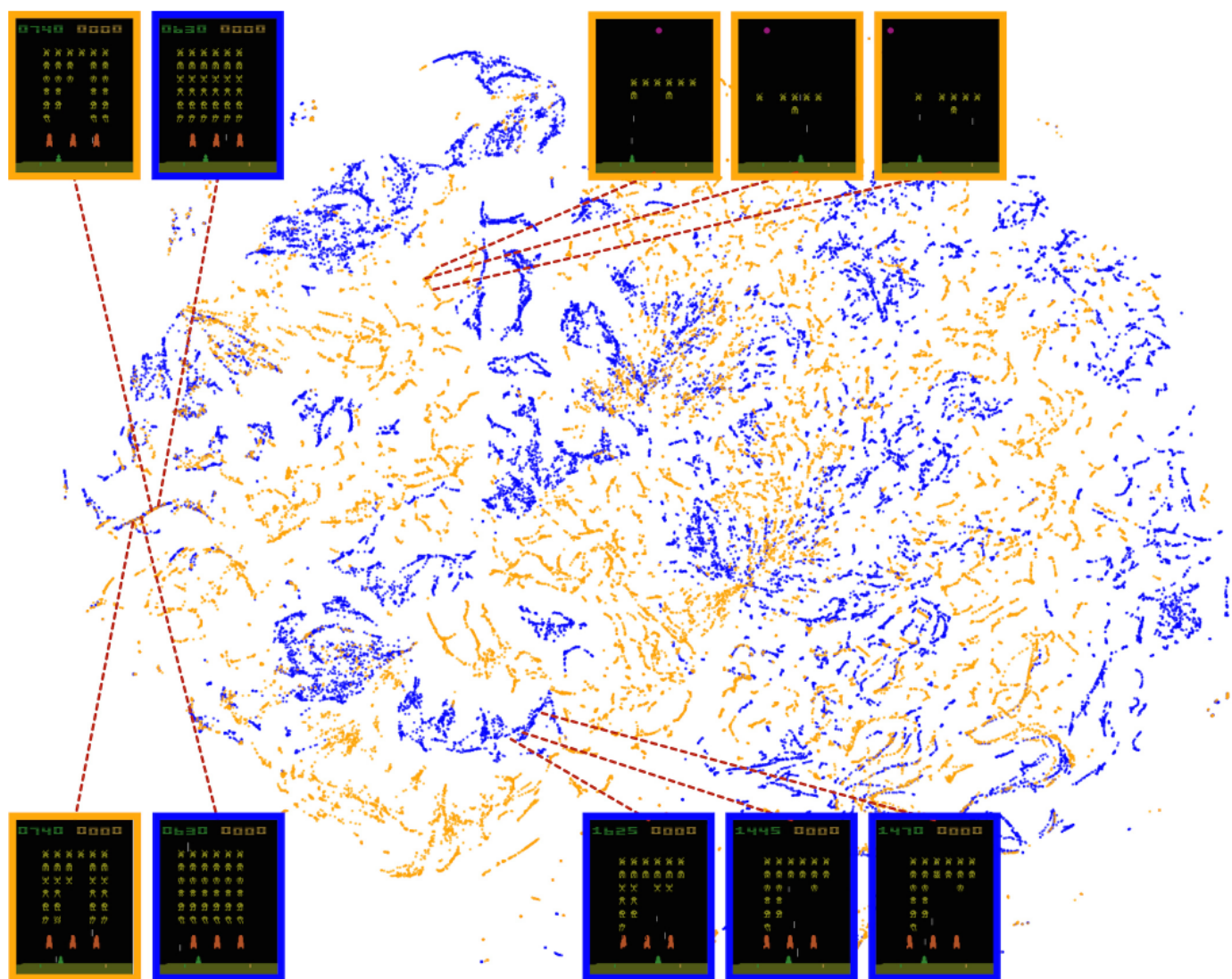
Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ with respect to the network parameters θ

Every C steps reset $\hat{Q} = Q$

结束于

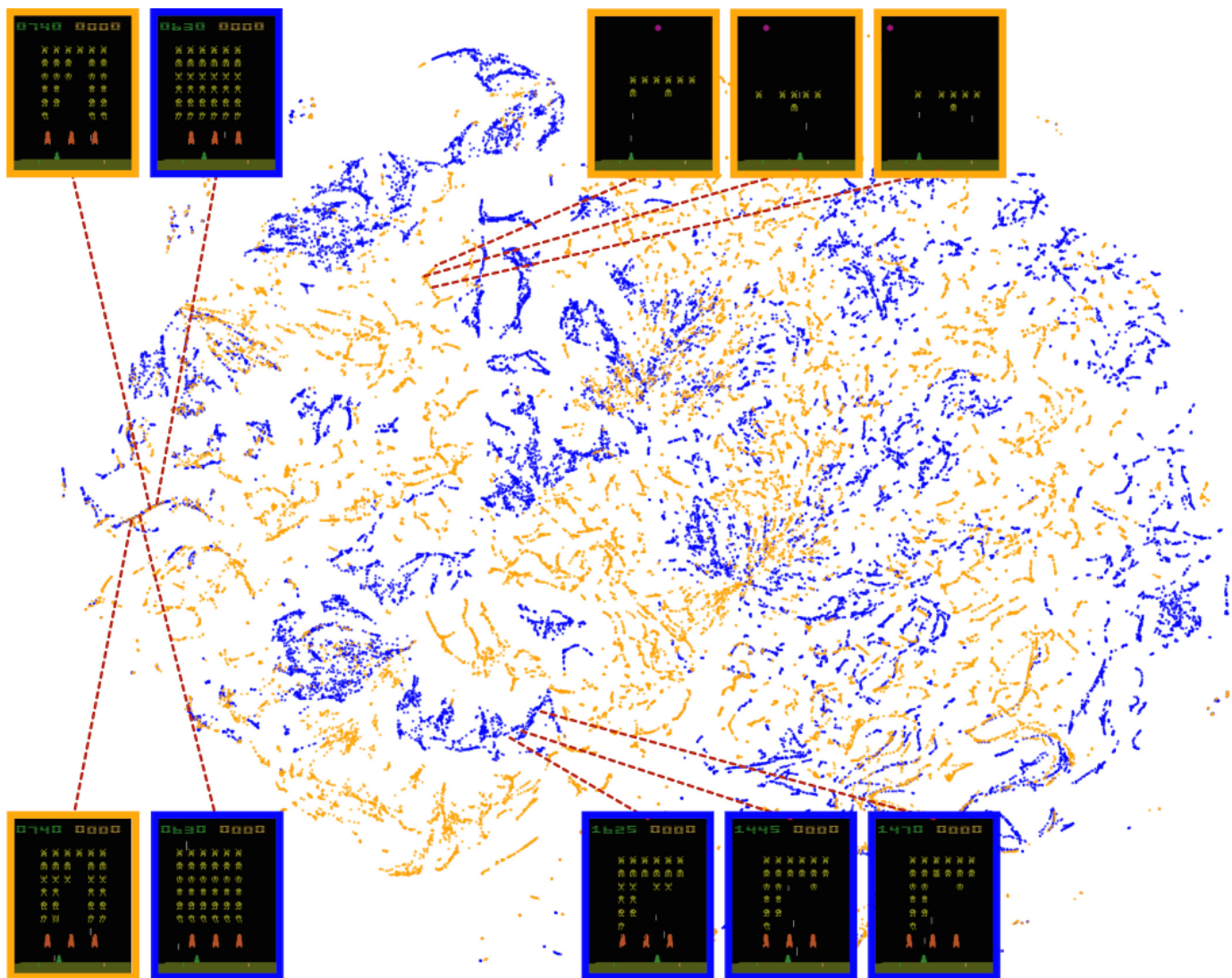
结束于

31. Jarrett, K., Kavukcuoglu, K., Ranzato, M. A. & LeCun, Y. 什么是物体识别的最佳多阶段架构? Proc. IEEE. Int. Conf. Comput. Vis. 2146–2153 (2009). 32. Nair, V. & Hinton, G. E. 整流线性单元改进了受限玻尔兹曼机。过程。国际。会议。马赫。学习。807–814 (2010)。33. Kaelbling, L. P., Littman, M. L. 和 Cassandra, A. R. 在部分可观察的随机域中进行规划和行动。人工智能 101, 99–134 (1994)。



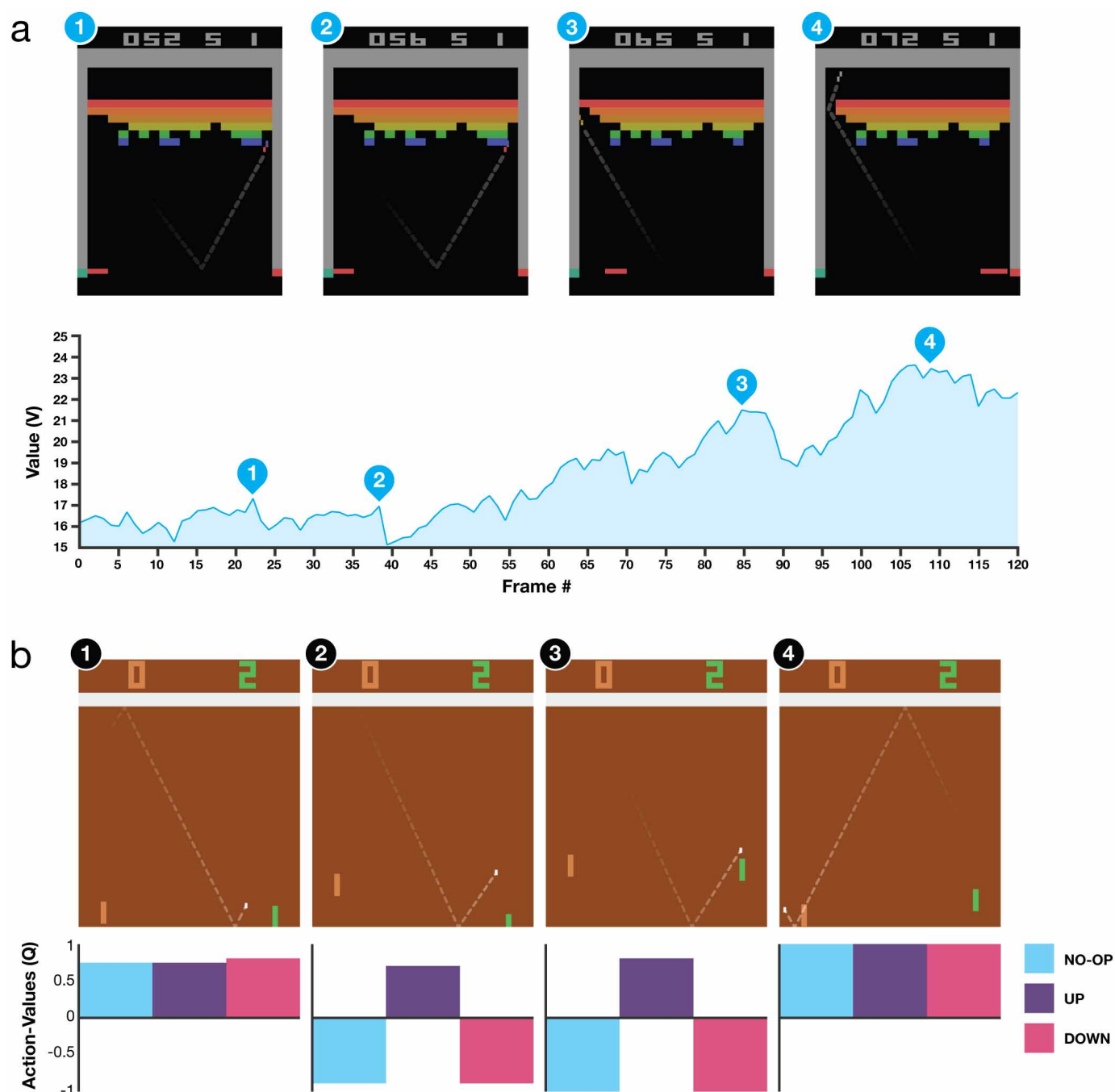
Extended Data Figure 1 | Two-dimensional t-SNE embedding of the representations in the last hidden layer assigned by DQN to game states experienced during a combination of human and agent play in Space Invaders. The plot was generated by running the t-SNE algorithm²⁵ on the last hidden layer representation assigned by DQN to game states experienced during a combination of human (30 min) and agent (2 h) play. The fact that there is similar structure in the two-dimensional embeddings corresponding to the DQN representation of states experienced during human play (orange

points) and DQN play (blue points) suggests that the representations learned by DQN do indeed generalize to data generated from policies other than its own. The presence in the t-SNE embedding of overlapping clusters of points corresponding to the network representation of states experienced during human and agent play shows that the DQN agent also follows sequences of states similar to those found in human play. Screenshots corresponding to selected states are shown (human: orange border; DQN: blue border).



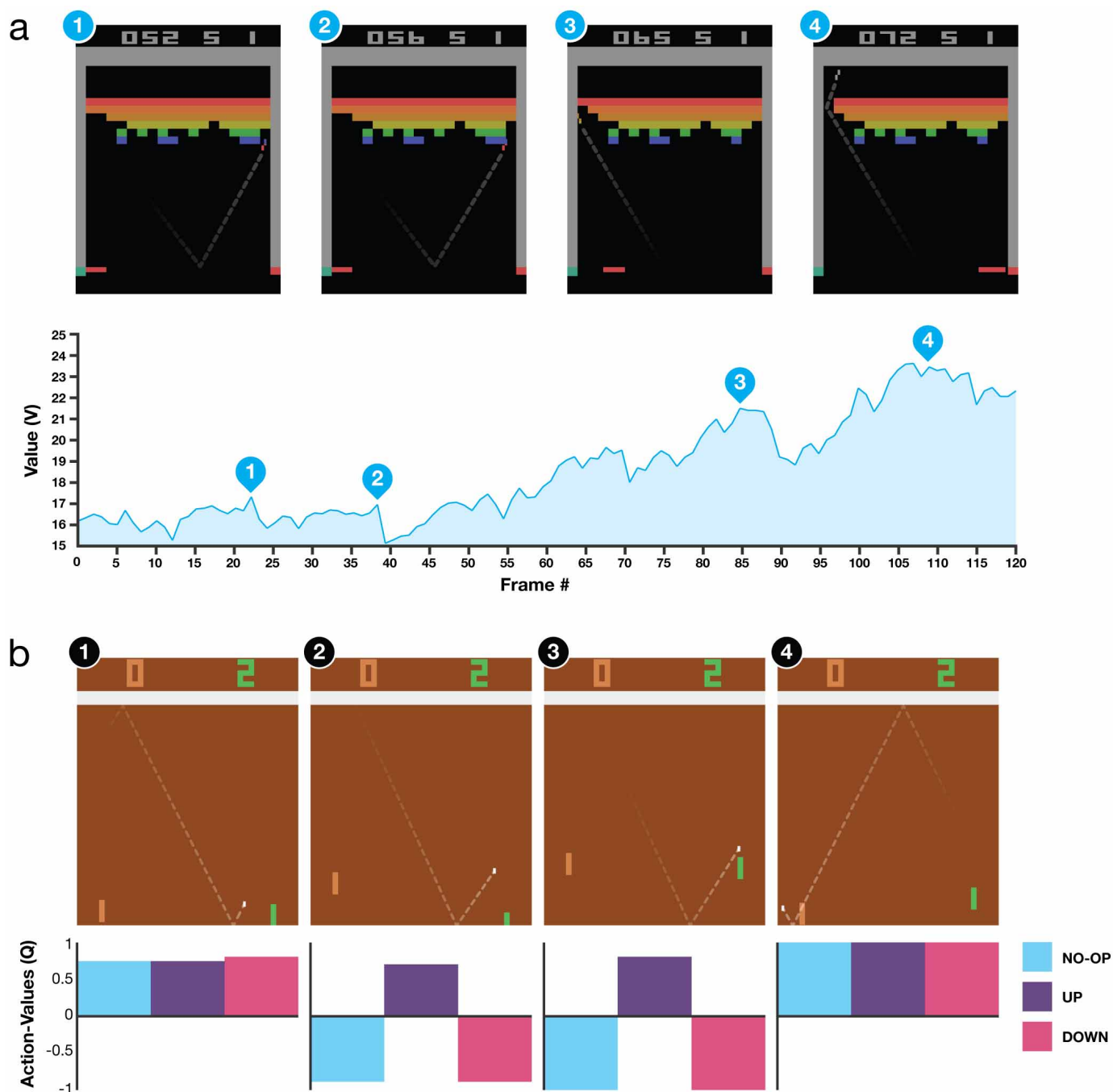
扩展数据图 1 | 由 DQN 分配的最后一个隐藏层中的表示的二维 t-SNE 嵌入，用于在《太空入侵者》中人类和智能体玩游戏时所经历的游戏状态。该图是通过在 DQN 指定的最后一个隐藏层表示上运行 t-SNE 算法²⁵来生成的，该表示表示人类（30 分钟）和智能体（2 小时）游戏过程中所经历的游戏状态。事实上，与人类游戏期间经历的状态的 DQN 表示相对应的二维嵌入中存在类似的结构（橙色

点）和 DQN 游戏（蓝点）表明，DQN 学习到的表示确实可以推广到从策略生成的数据，而不是它自己的策略。t-SNE 嵌入中存在与人类和智能体游戏期间经历的状态的网络表示相对应的重叠点簇，这表明 DQN 智能体也遵循与人类游戏中发现的状态序列类似的状态序列。显示与所选状态相对应的屏幕截图（人类：橙色边框；DQN：蓝色边框）。



Extended Data Figure 2 | Visualization of learned value functions on two games, Breakout and Pong. **a**, A visualization of the learned value function on the game Breakout. At time points 1 and 2, the state value is predicted to be ~ 17 and the agent is clearing the bricks at the lowest level. Each of the peaks in the value function curve corresponds to a reward obtained by clearing a brick. At time point 3, the agent is about to break through to the top level of bricks and the value increases to ~ 21 in anticipation of breaking out and clearing a large set of bricks. At point 4, the value is above 23 and the agent has broken through. After this point, the ball will bounce at the upper part of the bricks clearing many of them by itself. **b**, A visualization of the learned action-value function on the game Pong. At time point 1, the ball is moving towards the paddle controlled by the agent on the right side of the screen and the values of

all actions are around 0.7, reflecting the expected value of this state based on previous experience. At time point 2, the agent starts moving the paddle towards the ball and the value of the 'up' action stays high while the value of the 'down' action falls to -0.9 . This reflects the fact that pressing 'down' would lead to the agent losing the ball and incurring a reward of -1 . At time point 3, the agent hits the ball by pressing 'up' and the expected reward keeps increasing until time point 4, when the ball reaches the left edge of the screen and the value of all actions reflects that the agent is about to receive a reward of 1. Note, the dashed line shows the past trajectory of the ball purely for illustrative purposes (that is, not shown during the game). With permission from Atari Interactive, Inc.



扩展数据图 2 | 两种游戏 (Breakout 和 Pong) 的学习价值函数的可视化。a, Breakout 游戏中学习到的价值函数的可视化。在时间点 1 和 2, 状态值预计为 .17 并且智能体正在清除最低级别的砖块。价值函数曲线中的每个峰值对应于清除砖块所获得的奖励。在时间点 3, 代理即将突破到砖块的顶层, 并且值增加到 .21, 预计会突破并清除一大堆砖块。在第 4 点, 该值高于 23, 代理已突破。在此之后, 球将在砖块的上部弹起, 自行清除其中的许多砖块。b, Pong 游戏中学习到的动作价值函数的可视化。在时间点 1, 球朝屏幕右侧代理控制的球板移动, 并且值

所有动作都在 0.7 左右, 反映了根据以往经验对该状态的预期值。在时间点 2, 智能体开始将球拍移向球, “向上”动作的值保持较高水平, 而“向下”动作的值则降至 20.9。这反映出按下“向下”会导致智能体失去球并获得 21 的奖励。在时间点 3, 智能体通过按下“向上”击球, 预期奖励不断增加, 直到时间点 4, 此时球到达屏幕的左边缘, 所有动作的值反映智能体即将获得 1 的奖励。请注意, 虚线显示了球过去的轨迹, 纯粹用于说明目的 (即, 在游戏过程中未显示)。经 Atari Interactive, Inc. 许可。

Extended Data Table 1 | List of hyperparameters and their values

Hyperparameter	Value	Description
minibatch size	32	Number of training cases over which each stochastic gradient descent (SGD) update is computed.
replay memory size	1000000	SGD updates are sampled from this number of most recent frames.
agent history length	4	The number of most recent frames experienced by the agent that are given as input to the Q network.
target network update frequency	10000	The frequency (measured in the number of parameter updates) with which the target network is updated (this corresponds to the parameter C from Algorithm 1).
discount factor	0.99	Discount factor gamma used in the Q-learning update.
action repeat	4	Repeat each action selected by the agent this many times. Using a value of 4 results in the agent seeing only every 4th input frame.
update frequency	4	The number of actions selected by the agent between successive SGD updates. Using a value of 4 results in the agent selecting 4 actions between each pair of successive updates.
learning rate	0.00025	The learning rate used by RMSProp.
gradient momentum	0.95	Gradient momentum used by RMSProp.
squared gradient momentum	0.95	Squared gradient (denominator) momentum used by RMSProp.
min squared gradient	0.01	Constant added to the squared gradient in the denominator of the RMSProp update.
initial exploration	1	Initial value of ϵ in ϵ -greedy exploration.
final exploration	0.1	Final value of ϵ in ϵ -greedy exploration.
final exploration frame	1000000	The number of frames over which the initial value of ϵ is linearly annealed to its final value.
replay start size	50000	A uniform random policy is run for this number of frames before learning starts and the resulting experience is used to populate the replay memory.
no-op max	30	Maximum number of "do nothing" actions to be performed by the agent at the start of an episode.

The values of all the hyperparameters were selected by performing an informal search on the games Pong, Breakout, Seaquest, Space Invaders and Beam Rider. We did not perform a systematic grid search owing to the high computational cost, although it is conceivable that even better results could be obtained by systematically tuning the hyperparameter values.

扩展数据表1 超参数及其值列表

Hyperparameter	Value	Description
minibatch size	32	Number of training cases over which each stochastic gradient descent (SGD) update is computed.
replay memory size	1000000	SGD updates are sampled from this number of most recent frames.
agent history length	4	The number of most recent frames experienced by the agent that are given as input to the Q network.
target network update frequency	10000	The frequency (measured in the number of parameter updates) with which the target network is updated (this corresponds to the parameter <i>C</i> from Algorithm 1).
discount factor	0.99	Discount factor gamma used in the Q-learning update.
action repeat	4	Repeat each action selected by the agent this many times. Using a value of 4 results in the agent seeing only every 4th input frame.
update frequency	4	The number of actions selected by the agent between successive SGD updates. Using a value of 4 results in the agent selecting 4 actions between each pair of successive updates.
learning rate	0.00025	The learning rate used by RMSProp.
gradient momentum	0.95	Gradient momentum used by RMSProp.
squared gradient momentum	0.95	Squared gradient (denominator) momentum used by RMSProp.
min squared gradient	0.01	Constant added to the squared gradient in the denominator of the RMSProp update.
initial exploration	1	Initial value of ϵ in ϵ -greedy exploration.
final exploration	0.1	Final value of ϵ in ϵ -greedy exploration.
final exploration frame	1000000	The number of frames over which the initial value of ϵ is linearly annealed to its final value.
replay start size	50000	A uniform random policy is run for this number of frames before learning starts and the resulting experience is used to populate the replay memory.
no-op max	30	Maximum number of "do nothing" actions to be performed by the agent at the start of an episode.

超参数的值是通过在 Pong、Breakout、Seaquest、Space Invaders 和 BeamRider 游戏上进行非正式搜索来选择的。由于计算成本较高，我们没有进行系统的网格搜索，尽管可以想象，通过系统地调整超参数值可以获得更好的结果。

Extended Data Table 2 | Comparison of games scores obtained by DQN agents with methods from the literature^{12,15} and a professional human games tester

Game	Random Play	Best Linear Learner	Contingency (SARSA)	Human	DQN ($\pm$ std)	Normalized DQN (% Human)
Alien	227.8	939.2	103.2	6875	3069 ($\pm$ 1093)	42.7%
Amidar	5.8	103.4	183.6	1676	739.5 ($\pm$ 3024)	43.9%
Assault	222.4	628	537	1496	3359($\pm$ 775)	246.2%
Asterix	210	987.3	1332	8503	6012 ($\pm$ 1744)	70.0%
Asteroids	719.1	907.3	89	13157	1629 ($\pm$ 542)	7.3%
Atlantis	12850	62687	852.9	29028	85641($\pm$ 17600)	449.9%
Bank Heist	14.2	190.8	67.4	734.4	429.7 ($\pm$ 650)	57.7%
Battle Zone	2360	15820	16.2	37800	26300 ($\pm$ 7725)	67.6%
Beam Rider	363.9	929.4	1743	5775	6846 ($\pm$ 1619)	119.8%
Bowling	23.1	43.9	36.4	154.8	42.4 ($\pm$ 88)	14.7%
Boxing	0.1	44	9.8	4.3	71.8 ($\pm$ 8.4)	1707.9%
Breakout	1.7	5.2	6.1	31.8	401.2 ($\pm$ 26.9)	1327.2%
Centipede	2091	8803	4647	11963	8309($\pm$ 5237)	63.0%
Chopper Command	811	1582	16.9	9882	6687 ($\pm$ 2916)	64.8%
Crazy Climber	10781	23411	149.8	35411	114103 ($\pm$ 22797)	419.5%
Demon Attack	152.1	520.5	0	3401	9711 ($\pm$ 2406)	294.2%
Double Dunk	-18.6	-13.1	-16	-15.5	-18.1 ($\pm$ 2.6)	17.1%
Enduro	0	129.1	159.4	309.6	301.8 ($\pm$ 24.6)	97.5%
Fishing Derby	-91.7	-89.5	-85.1	5.5	-0.8 ($\pm$ 19.0)	93.5%
Freeway	0	19.1	19.7	29.6	30.3 ($\pm$ 0.7)	102.4%
Frostbite	65.2	216.9	180.9	4335	328.3 ($\pm$ 250.5)	6.2%
Gopher	257.6	1288	2368	2321	8520 ($\pm$ 3279)	400.4%
Gravitar	173	387.7	429	2672	306.7 ($\pm$ 223.9)	5.3%
H.E.R.O.	1027	6459	7295	25763	19950 ($\pm$ 158)	76.5%
Ice Hockey	-11.2	-9.5	-3.2	0.9	-1.6 ($\pm$ 2.5)	79.3%
James Bond	29	202.8	354.1	406.7	576.7 ($\pm$ 175.5)	145.0%
Kangaroo	52	1622	8.8	3035	6740 ($\pm$ 2959)	224.2%
Krull	1598	3372	3341	2395	3805 ($\pm$ 1033)	277.0%
Kung-Fu Master	258.5	19544	29151	22736	23270 ($\pm$ 5955)	102.4%
Montezuma's Revenge	0	10.7	259	4367	0 ($\pm$ 0)	0.0%
Ms. Pacman	307.3	1692	1227	15693	2311($\pm$ 525)	13.0%
Name This Game	2292	2500	2247	4076	7257 ($\pm$ 547)	278.3%
Pong	-20.7	-19	-17.4	9.3	18.9 ($\pm$ 1.3)	132.0%
Private Eye	24.9	684.3	86	69571	1788 ($\pm$ 5473)	2.5%
Q*Bert	163.9	613.5	960.3	13455	10596 ($\pm$ 3294)	78.5%
River Raid	1339	1904	2650	13513	8316 ($\pm$ 1049)	57.3%
Road Runner	11.5	67.7	89.1	7845	18257 ($\pm$ 4268)	232.9%
Robotank	2.2	28.7	12.4	11.9	51.6 ($\pm$ 4.7)	509.0%
Seaquest	68.4	664.8	675.5	20182	5286($\pm$ 1310)	25.9%
Space Invaders	148	250.1	267.9	1652	1976 ($\pm$ 893)	121.5%
Star Gunner	664	1070	9.4	10250	57997 ($\pm$ 3152)	598.1%
Tennis	-23.8	-0.1	0	-8.9	-2.5 ($\pm$ 1.9)	143.2%
Time Pilot	3568	3741	24.9	5925	5947 ($\pm$ 1600)	100.9%
Tutankham	11.4	114.3	98.2	167.6	186.7 ($\pm$ 41.9)	112.2%
Up and Down	533.4	3533	2449	9082	8456 ($\pm$ 3162)	92.7%
Venture	0	66	0.6	1188	3800 ($\pm$ 238.6)	32.0%
Video Pinball	16257	16871	19761	17298	42684 ($\pm$ 16287)	2539.4%
Wizard of Wor	563.5	1981	36.9	4757	3393 ($\pm$ 2019)	67.5%
Zaxxon	32.5	3365	21.4	9173	4977 ($\pm$ 1235)	54.1%

Best Linear Learner is the best result obtained by a linear function approximator on different types of hand designed features¹². Contingency (SARSA) agent figures are the results obtained in ref. 15. Note the figures in the last column indicate the performance of DQN relative to the human games tester, expressed as a percentage, that is, $100 \times (\text{DQN score} - \text{random play score})/(\text{human score} - \text{random play score})$.

延伸数据表2 | DQN 代理与文献方法^{12,15} 和专业方法获得的游戏分数的比较
胡马报 游戏测试员

Game	Random Play	Best Linear Learner	Contingency (SARSA)	Human	DQN ($\pm$ std)	Normalized DQN (% Human)
Alien	227.8	939.2	103.2	6875	3069 ($\pm$ 1093)	42.7%
Amidar	5.8	103.4	183.6	1676	739.5 ($\pm$ 3024)	43.9%
Assault	222.4	628	537	1496	3359($\pm$ 775)	246.2%
Asterix	210	987.3	1332	8503	6012 ($\pm$ 1744)	70.0%
Asteroids	719.1	907.3	89	13157	1629 ($\pm$ 542)	7.3%
Atlantis	12850	62687	852.9	29028	85641($\pm$ 17600)	449.9%
Bank Heist	14.2	190.8	67.4	734.4	429.7 ($\pm$ 650)	57.7%
Battle Zone	2360	15820	16.2	37800	26300 ($\pm$ 7725)	67.6%
Beam Rider	363.9	929.4	1743	5775	6846 ($\pm$ 1619)	119.8%
Bowling	23.1	43.9	36.4	154.8	42.4 ($\pm$ 88)	14.7%
Boxing	0.1	44	9.8	4.3	71.8 ($\pm$ 8.4)	1707.9%
Breakout	1.7	5.2	6.1	31.8	401.2 ($\pm$ 26.9)	1327.2%
Centipede	2091	8803	4647	11963	8309($\pm$ 5237)	63.0%
Chopper Command	811	1582	16.9	9882	6687 ($\pm$ 2916)	64.8%
Crazy Climber	10781	23411	149.8	35411	114103 ($\pm$ 22797)	419.5%
Demon Attack	152.1	520.5	0	3401	9711 ($\pm$ 2406)	294.2%
Double Dunk	-18.6	-13.1	-16	-15.5	-18.1 ($\pm$ 2.6)	17.1%
Enduro	0	129.1	159.4	309.6	301.8 ($\pm$ 24.6)	97.5%
Fishing Derby	-91.7	-89.5	-85.1	5.5	-0.8 ($\pm$ 19.0)	93.5%
Freeway	0	19.1	19.7	29.6	30.3 ($\pm$ 0.7)	102.4%
Frostbite	65.2	216.9	180.9	4335	328.3 ($\pm$ 250.5)	6.2%
Gopher	257.6	1288	2368	2321	8520 ($\pm$ 3279)	400.4%
Gravitar	173	387.7	429	2672	306.7 ($\pm$ 223.9)	5.3%
H.E.R.O.	1027	6459	7295	25763	19950 ($\pm$ 158)	76.5%
Ice Hockey	-11.2	-9.5	-3.2	0.9	-1.6 ($\pm$ 2.5)	79.3%
James Bond	29	202.8	354.1	406.7	576.7 ($\pm$ 175.5)	145.0%
Kangaroo	52	1622	8.8	3035	6740 ($\pm$ 2959)	224.2%
Krull	1598	3372	3341	2395	3805 ($\pm$ 1033)	277.0%
Kung-Fu Master	258.5	19544	29151	22736	23270 ($\pm$ 5955)	102.4%
Montezuma's Revenge	0	10.7	259	4367	0 ($\pm$ 0)	0.0%
Ms. Pacman	307.3	1692	1227	15693	2311($\pm$ 525)	13.0%
Name This Game	2292	2500	2247	4076	7257 ($\pm$ 547)	278.3%
Pong	-20.7	-19	-17.4	9.3	18.9 ($\pm$ 1.3)	132.0%
Private Eye	24.9	684.3	86	69571	1788 ($\pm$ 5473)	2.5%
Q*Bert	163.9	613.5	960.3	13455	10596 ($\pm$ 3294)	78.5%
River Raid	1339	1904	2650	13513	8316 ($\pm$ 1049)	57.3%
Road Runner	11.5	67.7	89.1	7845	18257 ($\pm$ 4268)	232.9%
Robotank	2.2	28.7	12.4	11.9	51.6 ($\pm$ 4.7)	509.0%
Seaquest	68.4	664.8	675.5	20182	5286($\pm$ 1310)	25.9%
Space Invaders	148	250.1	267.9	1652	1976 ($\pm$ 893)	121.5%
Star Gunner	664	1070	9.4	10250	57997 ($\pm$ 3152)	598.1%
Tennis	-23.8	-0.1	0	-8.9	-2.5 ($\pm$ 1.9)	143.2%
Time Pilot	3568	3741	24.9	5925	5947 ($\pm$ 1600)	100.9%
Tutankham	11.4	114.3	98.2	167.6	186.7 ($\pm$ 41.9)	112.2%
Up and Down	533.4	3533	2449	9082	8456 ($\pm$ 3162)	92.7%
Venture	0	66	0.6	1188	3800 ($\pm$ 238.6)	32.0%
Video Pinball	16257	16871	19761	17298	42684 ($\pm$ 16287)	2539.4%
Wizard of Wor	563.5	1981	36.9	4757	3393 ($\pm$ 2019)	67.5%
Zaxxon	32.5	3365	21.4	9173	4977 ($\pm$ 1235)	54.1%

最佳线性函数逼近器在不同类型的手工设计特征¹²上获得的最佳结果。应急 (SARSA) 人员数字是在参考文献中获得的结果。15. 注意
字我 n 最后一列表示DQN相对于人类游戏测试者的表现，以百分比表示，即100.3(DQN得分/2随机游戏得分)/(人类得分/2随机游戏得分)

数
关于)。

Extended Data Table 3 | The effects of replay and separating the target Q-network

Game	With replay, with target Q	With replay, without target Q	Without replay, with target Q	Without replay, without target Q
Breakout	316.8	240.7	10.2	3.2
Enduro	1006.3	831.4	141.9	29.1
River Raid	7446.6	4102.8	2867.7	1453.0
Seaquest	2894.4	822.6	1003.0	275.8
Space Invaders	1088.9	826.3	373.2	302.0

DQN agents were trained for 10 million frames using standard hyperparameters for all possible combinations of turning replay on or off, using or not using a separate target Q-network, and three different learning rates. Each agent was evaluated every 250,000 training frames for 135,000 validation frames and the highest average episode score is reported. Note that these evaluation episodes were not truncated at 5 min leading to higher scores on Enduro than the ones reported in Extended Data Table 2. Note also that the number of training frames was shorter (10 million frames) as compared to the main results presented in Extended Data Table 2 (50 million frames).

扩展数据表3 | 重放和分离目标Q网络的效果

Game	With replay, with target Q	With replay, without target Q	Without replay, with target Q	Without replay, without target Q
Breakout	316.8	240.7	10.2	3.2
Enduro	1006.3	831.4	141.9	29.1
River Raid	7446.6	4102.8	2867.7	1453.0
Seaquest	2894.4	822.6	1003.0	275.8
Space Invaders	1088.9	826.3	373.2	302.0

使用标准超参数对 DQN 代理进行 1000 万帧训练，以实现打开或关闭重放的所有可能组合、使用或不使用单独的目标 Q 网络以及三种不同的学习率。每 250,000 个训练帧对每个代理进行 135,000 个验证帧的评估，并报告最高平均情节分数。请注意，这些评估片段没有在 5 分钟时被截断，从而导致 Enduro 上的分数高于扩展数据表 2 中报告的分数。另请注意，与扩展数据表 2 中提供的主要结果（5000 万帧）相比，训练帧的数量更短（1000 万帧）。

Extended Data Table 4 | Comparison of DQN performance with linear function approximator

Game	DQN	Linear
Breakout	316.8	3.00
Enduro	1006.3	62.0
River Raid	7446.6	2346.9
Seaquest	2894.4	656.9
Space Invaders	1088.9	301.3

The performance of the DQN agent is compared with the performance of a linear function approximator on the 5 validation games (that is, where a single linear layer was used instead of the convolutional network, in combination with replay and separate target network). Agents were trained for 10 million frames using standard hyperparameters, and three different learning rates. Each agent was evaluated every 250,000 training frames for 135,000 validation frames and the highest average episode score is reported. Note that these evaluation episodes were not truncated at 5 min leading to higher scores on Enduro than the ones reported in Extended Data Table 2. Note also that the number of training frames was shorter (10 million frames) as compared to the main results presented in Extended Data Table 2 (50 million frames).

扩展数据表 4 | DQN 性能与线性函数逼近器的比较

Game	DQN	Linear
Breakout	316.8	3.00
Enduro	1006.3	62.0
River Raid	7446.6	2346.9
Seaquest	2894.4	656.9
Space Invaders	1088.9	301.3

DQN 代理的性能与线性函数逼近器在 5 个验证游戏中的性能进行了比较（即，使用单个线性层代替卷积网络，并结合重放和单独的目标网络）。使用标准超参数和三种不同的学习率对代理进行了 1000 万帧的训练。每 250,000 个训练帧对每个代理进行 135,000 个验证帧的评估，并报告最高平均情节分数。请注意，这些评估片段没有在 5 分钟时被截断，从而导致 Enduro 上的分数高于扩展数据表 2 中报告的分数。另请注意，与扩展数据表 2 中提供的主要结果（5000 万帧）相比，训练帧的数量更短（1000 万帧）。